
The Complete OpenSuSE Mail Server

... or how to really understand a full IMAP/SMTP/HTTPS/Webmail setup

Johan Persson <johan162@gmail.com>

Version 0.1

Copyright © 2011 Johan Persson

Table of Contents

1. Introduction	3
1.1. Why another mail tutorial?	3
1.2. Applicable OpenSuSE versions and pre-requisites	4
1.3. Typographical conventions used in this tutorial	7
1.4. About this document	7
2. Overview of the components	9
2.1. Postfix	9
2.2. Dovecot	11
2.3. Fetchmail	12
2.4. Procmail	12
2.5. Roundcube (and Apache)	13
2.6. openssl	14
2.7. spamassassin	14
3. The setup process	16
3.1. Some ports and acronyms used	16
3.2. Using SSL/TLS certificates	18
4. Setting up Apache HTTP-server	23
4.1. Creating self-signed SSL/TLS certificate for Apache	23
4.2. Enabling SSL in the OpenSuSE configuration	24
4.3. Setting up a virtual host for HTTPS	25
4.4. Restarting and enabling Apache	26
5. Setting up Dovecot	28
5.1. Modifying dovecot configuration files	28
5.2. Creating dovecot SSL/TLS certificates	30
5.3. Testing the dovecot configuration	31
5.4. Verifying the encrypted IMAP connection	32
5.5. Troubleshooting Dovecot	32
5.6. Adding a full text index	33
6. Setting up postfix	34
6.1. User authentication in postfix	34
6.2. Setting up SSL/TLS for postfix	38
6.3. Enabling postfix at boot time	42
7. Setting up fetchmail	43
7.1. Configuring fetchmail	43
7.2. Starting and enabling fetchmail at boot time	44
8. Setting up spamassassin	45
8.1. Configuring spamassassin Bayes auto learning	45
8.2. Verifying spamassassin	46
9. Setting up procmail	48
9.1. Configuration file	48
10. Setting up Roundcube	51
11. Where to find more information	53
11.1. On the usage of SuSE Yast tool	53
11.2. References	53
A. Postfix configuration examples	54

A.1. Example 1	54
A.2. Example 2	54
A.3. Example 3	54
A.4. Example 4	55
B. Procmail recipes	55
B.1. The problem of filtering on the “Subject:” field	56
B.2. How to create filter that matches on any of several rules (creating an “OR” rule)	57
B.3. Out of office	57
B.4. Replying with a mail based on subject	57
B.5. Calling an arbitrary external program	58
B.6. A template to start with	58
C. Dovecot tips and tricks	59
C.1. Automatic expunging of old mails	59
C.2. Synchronizing (or backing up) two IMAP servers	60
D. Postfix configuration	60
D.1. main.cf	61
D.2. master.cf	65
E. Create SSL/TLS keys and certificates with openssl	65
E.1. The use of the term “self-signed certificates”	66
E.2. Preparations	67
E.3. Generating the CA key-pair	69
E.4. Generating the Apache HTTPS key-pair	71
E.5. Generating the Dovecot/postfix key-pair	71
E.6. Generating a certificate with multiple common names	72
E.7. Keeping both the certificate and key in the same file	73
E.8. Converting to other certificate formats	73
E.8.1. DER format	73
E.8.2. PKCS #12 format	74
E.9. Certification Revocation List (CRL) and OCSP	74
E.10. Summary	75
E.10.1. Creating the CA (root certificate)	75
E.10.2. Creating the Apache HTTPS certificate	75
E.10.3. Creating the mail certificate	75
F. Apache virtual host example configuration	76
F.1. ssl-vhost.conf	76

1 Introduction

With the almost ubiquitous presence of mobile devices it becomes crucial to be able to access mail from any device anywhere and still read and manipulate the same mails regardless of which device is used. The easiest way to achieve this with close to zero setup effort is to use a public mail provider, for example GMail, Hotmail etc.

However for users that require more fine grained control over message handling a public mail provider might not be a viable option. Reasons that might require such a setup includes but are not limited to:

- need for automatic processing of messages with external programs at message arrival (such as automatic replies depending on message content or perform other pre-defined actions such as scanning messages for viruses)
- control over mail backup
- collection of mails from several accounts into one master repository
- unable to comply with the license of the public mail provider
- need for an ad-free mail client
- needs mail to be sent from a specific domain
- privacy concerns

The rest of this tutorial will be spent going through, in details, how to make such a setup based on a OpenSuSE Linux/GNU distribution.

1.1 Why another mail tutorial?

There is no shortage of various notes and tutorials trying to show how to do a server setup like this. Our experience is however that many of them, while working (in some respect), have one or more misunderstandings or present obsoleted knowledge based on old ways of doing things. In many cases recommendations are handed down from older tutorial and seen as truth - because they work - but are incomplete or in worst case have been deprecated since long.

This tutorial, while substantially longer, aims at describing a correct and modern setup as well as how it works and how the components interact. Many misunderstanding stem from a poor grasp of the overall picture on how these component fits together. It is our belief that in order to configure a working setup one must understand what one is doing. Just taking random bits of code from "the net" might give, by sheer luck, a working setup but it might as likely have created an open relay for spammers to use (which is very bad).

Especially the SSL/TLS setup using openssl will be covered and explained in more details than usual in descriptions like this since much information available on "the net" is just plain wrong or at best shows very, very, antiquated ways of configuring a SSL/TLS setup.

Goals with this tutorial

The setup we will describe in this tutorial will allow any device capable of IMAP message handling (all modern devices do) to connect and synchronize mails from a central repository meaning that any change from any device will be visible to all other devices. The setup will also allow clients to send mails through this server (acting as an SMTPD server).

The setup assumes a "home server" which is not directly connected to the internet and configured as an official mail server for some domain. This also implies the need to relay mails to be sent through a "real" mail server (most likely an ISPs server) that is properly connected to the internet. An overview of the principle of our setup and all constituting parts are shown in Figure 2, "Mail server overview".

The goal of this tutorial is therefore to describe, in a very detailed way, how to setup an OpenSuSE server to both act as an IMAP and SMTP server. In addition it will use encrypted communication using a SSL/TLS setup based

on the `openssl` library. This will allow secure mail access from any device (e.g. SmartPhones, Ipads, laptops etc.) using either the device's built-in mail client or optionally connected to our web based mail client.

1.2 Applicable OpenSuSE versions and pre-requisites

Much of the information here is generic and not specific to OpenSuSE. However, location and structure of specific configuration files are distribution dependent. This document has been verified on OpenSuSE 11.4 as well as 12.1. Most of the details given in this tutorial should also be valid on older 11.x OpenSuSE releases. OpenSuSE resembles the RedHat distribution in some respect but has enough differences of its own to not make it directly compatible. Other systems (for example Ubuntu) uses a rather different setup structure so while the principles still apply the details will not match.

One main difference noticeable between 11.4 and 12.1 is that in 12.1 the older SysV init file handling has been replaced with the more modern `systemd` init file handling. This has several advantages such as parallelizing startup scripts (hence reducing boot time). Fortunately there is a compatibility layer so that old `/etc/init.d/` startup scripts will work as usual and redirect the start/stop commands to `systemd` (see `systemd – boot faster and cleaner` [<http://news.opensuse.org/2011/12/22/systemd-%E2%80%93-boot-faster-and-cleaner-with-opensuse-12-1/>])



Tip

When installing OpenSuSE chose the desktop as "*Minimal Server Selection (Text Mode)*" as illustrated in Figure 1, "Choosing text server when installing OpenSuSE". Note that in order to get to this screen the installation must be run in manual mode and not automatic.

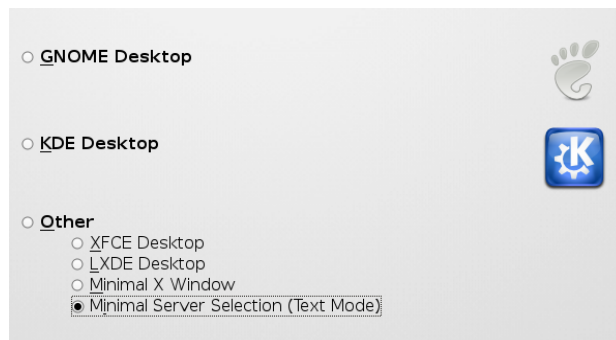


Figure 1: Choosing text server when installing OpenSuSE

Installed components

We will assume that all the components listed below are installed on the server. They are all available (with one exception as noted below) from the standard OpenSuSE repository as packages. For now it is enough if these components are installed but not configured - this is what the rest of this tutorial is all about. All these components can be installed from the Yast2 package manager as listed below (the "core packages" indicates the minimum needed packages.)

- **Apache v2.x** - HTTP server (will be referred to as only Apache in the rest of the document).

Core packages: `apache2`, `apache2-prefork`, `apache2-utils`, `libapr-util`, `libapr1`

- **PHP5** - With at least the modules `php5-curl`, `php5-openssl` and `php5-sockets` installed

Core packages: `apache2-mod_php5`, `php5-mysql`, `php5-curl`, `php5-openssl`, `php5-sockets`

- **Postfix** - Mail daemon used to receive and send mails.

Note: The default mail daemon (MTA) in OpenSuSE 12.1 has changed to *exim* if a full KDE/X-Windows installation is done). This means installing postfix might trigger a warning that it conflicts with *exim* and that it will be un-installed. Just answer yes in that case.

Core packages: `postfix`

- **Fetchmail** - Used to fetch messages from other mail accounts

Core packages: `fetchmail`

- **Dovecot v2.x** - IMAP server that allows access to your stored mails.

Note: Makes sure that Dovecot 2.x is installed and not Dovecot 1.x. OpenSuSE by default installs the older version 1.x of Dovecot. Dovecot 2.x is available in the OpenSuSE repositories (as `dovecot2`). We will simple refer to this as Dovecot.

Core packages: `dovecot20`

- **MySQL** - Database server (needed for the Roundcube webmailer)

Core packages: `mysql-community-server`, `mysql-community-server-client`

- **Procmail** - Used for rule based mail filtering which will allow things like sorting mails into folders depending on there content or do other types of automatic processing based on the content of the messages.

Core packages: `procmail`

- **Spamassassin** - Tool to help identify junk mails (spam)

Core packages: `spamassassin`, `perl-Mail-Spamassassin`

- **openssl** - Tool to handle encryption and creation of certificates (this package is installed by default in the Minimal Server Selection)

Core packages: `openssl`



Caution

Please make absolutely sure that all of the above components are installed before proceeding!

All necessary component can be installed by running the following command as root

```
$> zypper in apache2 apache2-prefork apache2-mod_php5 php5-curl \
php5-openssl php5-sockets postfix fetchmail dovecot20 procmail \
mysql-community-server mysql-community-server-client \
spamassassin perl-Mail-Spamassassin
```

There is also one final package that strictly speaking is not necessary but strongly recommended. This is to install a Web-based mail client. Having a web-based mail setup has the advantage that it will allow safe and secure access (using `https://`) to mails from any networked computer in the world. In this tutorial we will use "**Roundcube**" (<http://roundcube.net/>).

The application style of the modern (and AJAX based) Roundcube also makes it a compelling alternative even to native mail clients such as KMail. Roundcube is not as of this writing available as a package for OpenSuSE. However as a Web-applications it is straightforward to install and set it up using the included installer.

What level of experience is needed and how long will it take?

A certain maturity with Linux is assumed but mostly the ability (and willingness) to look out for details is what matters. As in a mathematical proof every single detail counts. There are literally hundreds of pages in the manuals for all the components we touch upon here so the details described here are only the details that really matters - the *need to know details*.

Setting up a complete IMAP/SMTP server correctly the first time requires an eye for details. Even experienced administrators will make mistakes. The main advantage with experience is that debugging various issues gets faster since likely problem spots are known by experience.

The setup of a working server will likely take several hours rather than minutes. So patience is needed as well as careful reading with a willingness to go back and double check everything at least three times!

As the installation progress make notes on what config files have been changed and what other actions has been performed. After a while it will simply be impossible to remember all the little details and what has been done and not.



Tip

We strongly encourage to install the server in a **virtual machine installation** (for example using VirtualBox) since that will easily allow going back to a known pristine state in case of "unrecoverable error". Compare this with having to re-install a physical server since one is not sure after a while which of all the settings that were changed was wrong.

Once a working server in a virtual machine is established it is also easy to move the virtual server, for example to a new server when the hardware is upgraded. This allows an easy upgrade path to future OpenSuSE version as well without having to reconfigure a complete mail server at each upgrade. When in the future OpenSuSE v13.1, v14.1, etc. comes out there is nothing that prevents running a virtual mail server based on 11.4 or 12.1. The core protocols of the internet are not very likely to have changed dramatically even by then so the virtual server will still be valid.

OpenSuSE has pre-built packages for VirtualBox which makes it straight forward to install.

The only minimal drawback with a virtual server is the need to reserve some RAM memory. However, given that it is perfectly fine to run a complete mail setup on 512MB (or less) this should be considered a small price to pay for the flexibility it will give.

1.3 Typographical conventions used in this tutorial

For special elements like configuration settings, commands, screen input/output etc. the convention shown in Table 1, “Typographic conventions” are used in this tutorial.

Table 1: Typographic conventions

Element	Display
File name	/etc/fetchmailrc
Directory name	/etc
Screen input/output	<pre>\$> /etc/init.d/apache2 start</pre>
	<i>Note: Screen output and listings may have additional line breaks indicated by a '\ ' character at the end of the line. Such line breaks should not be entered in the files or in the command sequences.</i>
Configuration file	<pre>poll remotemail.com proto pop3 user "joe" there \ with password "secret" is "joe" here;</pre>
Commands in text	Execute fetchmail once ...
Acronyms in text	The well known openssl library which supports SSL/TLS ...
Literal names in text	The antispam 501 line is used ...
Application name	The Roundcube mailer ...

In some cases the same name is used as either a command or as the name of an application. The context will determine if the name is used as a noun (name of application) or as verb (command).

The choice of formatting should also work well for the ~8% of the general population that suffer from color vision perception challenge (aka "color blindness") due to the use of both color and font change.

1.4 About this document

This tutorial was written in plain text format (UTF-8 encoding) using the **DocBook XML markup**. To render the styled output from the XML sources the **DocBook 1.77.1 XSL** stylesheets was used together with the **xsltproc** XSLT processor

To produce the PDF output the **DocBook XML** were first transformed into an FO format which was then rendered with the Apache FOP processor into the final PDF format. In order to get the current look and feel of the document a customized XSL stylesheet was used. To style the HTML output both a custom HTML XSL stylesheet and a CSS stylesheet was used.

As source editor oXygen XML editor was used and the transform process into HTML and PDF was controlled by a Phing script.

References for technologies used to produce this document

- **DocBook** [<http://docbook.org/>], is a defined XML schema that identifies the meaning of different parts of a document. It is specifically well suited to write technical computer related documents but can also be used to write generic documents. By itself DocBook does not care about how the document is displayed (in contrast to for example OpenOffice or Word).

In order to transform the DocBook XML document into something suitable for human reading (such as HTML and PDF) a set of XSL stylesheets are used. They are known as the DocBook XSL stylesheets. The current version as of this writing was 1.77.1

- XSL [<http://www.w3.org/Style/XSL/>], stands for [Definition: Extensible Stylesheet Language Family] is an overall term for three technologies used to process XML documents into other XML documents. The included technologies are:
 - XSLT [<http://www.w3.org/TR/xslt>], is a programming language for transforming XML documents into other well formed XML document, for example (X)HTML documents.
 - XPath [<http://www.w3.org/TR/xpath/>], a way to uniquely identify different parts of a well formed XML document. This is used as the search language in the stylesheet to specify what elements to transform.
 - XSL-FO [<http://www.w3.org/TR/xsl/>], is a defined XML schema that defines how a printed output should be displayed. This format can later be easily transformed into other formats suitable for printed output such as PDF or RTF.
- xsltproc [<http://xmlsoft.org/XSLT/xsltproc2.html>], is the GNOME projects XSLT processor that uses an XSL(T) program to transform one XML document into another. This is one of the few XSLT processors written in C (most others are written in Java, for example Saxon)
- fop [<http://xmlgraphics.apache.org/fop/>], is the Apache projects XSL-FO processor. This is primarily used to transform a XSL-FO document into a PDF document.
- Oxygen [<http://www.oxygenxml.com/>], the XML editor used to for writing the source XML document. Any text editor can of course be used to write XML but this editor has a lot of intelligence as well as a preview mode that lets you work with an approximation of how a typical HTML rendered version of the XML document will look which makes it much quicker to produce this type of XML documents.

2 Overview of the components

Unfortunately there isn't one simple program that will do all what we need. Instead we will need a number of components to achieve our goal. A server setup with IMAP/SMTP has some reputation of being “black-magic” to setup. However, this is mostly based on the fact that there are many different parts involved and misunderstanding the role of (or interactions between) the different actors will definitely make the process seem like black magic. So let's start by discussing the parts involved and give a high level logical overview on how the parts interact. For a pictorial overview please refer to Figure 2, “Mail server overview”

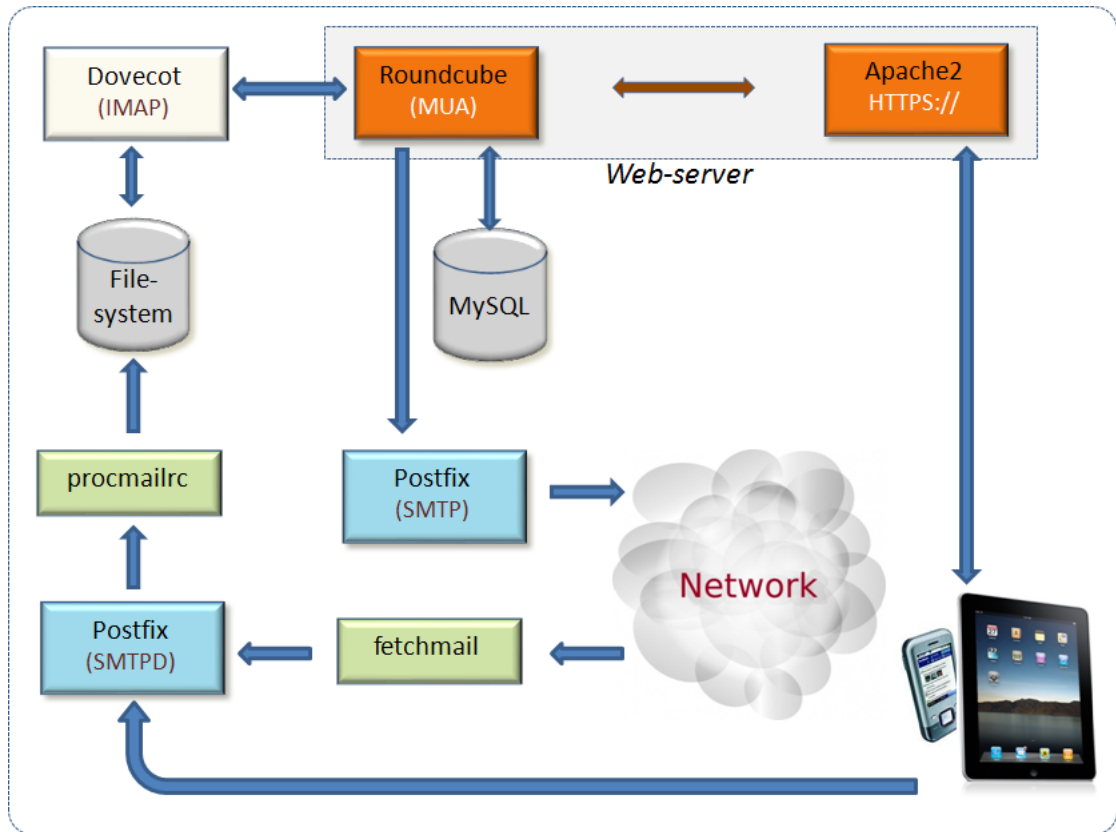


Figure 2: Mail server overview

2.1 Postfix

This is the SMTP/SMTPD server/daemon that handles the mail protocols we will be using. The Postfix server is abstractly known as an MTA (*Mail Transfer Agent*) in mail lingo. These are the backbones of message transfer over the internet. The role of the MTAs are to receive and forward messages to other MTAs depending on the end address. This forwarding continues until the message reaches the server that is responsible for the domain the message is addressed to. Once the message is at its final destination it gets delivered to the appropriate user in that domain.

The headers in a typical message will show which MTAs have been passed in the process of delivering the mail from the sender to the recipient.

Initially a user creates a message with some mail client, a mail client is abstractly known as MUA, (*Mail User Agent*) examples are (for example) KMail, Thunderbird or Roundcube. The MUA initially connects to a specified MTA that receives the message. The MTA examines the headers in the mail and determines if it should forward the message to some other MTA or if the message is to be locally delivered.

The reason for using this abstract language instead of referring to the specific programs we are using is to be able to discuss mails without referring to specific protocols or applications.

All of the above is relevant only to officially registered mail servers for a specific domain. These mail servers are recognized in the DNS system with a so called MX record (Mail eXchange). A home based server is normally not an officially recognized mail server responsible for a particular domain. Instead the home MTA has to be configured to relay messages it receives from a mail client on to an official mail server. This is normally the mail server at the ISP.



Note

For a proper email server connected to the internet with a MX (Mail eXchange) record in its DNS entry there are many more details to take into account. For example DKIM, FBL and SPF just to name a few techniques and methods a sysadmin has to be familiar with. Luckily for a home server we can ignore this.

Postfix has two separate and distinct roles, one as a sending server, an SMTP server, and one as a receiving daemon, the SMTPD server. The SMTP server is responsible for sending mail to other servers while the receiving server is a daemon that waits for connections from other servers (or clients) to receive messages. Any messages that should be delivered locally will be stored the users mailbox and any message with a recipient outside the domain the server is responsible for will be forwarded to the next mail server depending on the end address.

By default the assigned port that the receiving MTA server listens on for non-encrypted messages are on port 25, see figure Figure 3, “The dual role of postfix”. This is the port that is used both when other MTAs to talk to our server to forward messages as well as the port that mail clients connects on to initially send messages created by the client. (This is how most setups work in practice for historical reasons. However, this is not really in compliance with the original RFCs. In the original specifications port 25 should only be used for MTA to MTA communication and port 587 should be used when a mail clients want to send a new message to a MTA.)

An important part of this setup is that the server will relay all incoming mails to the ISPs mail server which will then send it to the rest of the world. Most ISP will not allow neither outgoing nor incoming traffic on port 25 to any other servers besides their own. This makes it impossible to either directly send messages on port 25 to our server from the outside or to use any other mail server besides the ISP to forward our messages to. The ISPs block port 25 in an futile attempt to block spams that are sent by Trojan mail servers running on unsuspecting servers. The reason this is futile is that it is as easy to use another port on the server for the spammers. The ISP who is blocking of port 25 is today mainly just an irritation for advanced users. It does not stop spam in any shape or form.

What we need to do in order to send mail via our home server is to use another port, connect to that other port and have postfix relay our message to be sent to our ISPs official mail server. This will of course assume that we have told our SMTPD daemon to listen on that port.

The reasons we do not have our mail clients connect directly to our ISPs mail server but instead going through our home server are twofold:

- most ISP does not allow connections to their mail server unless the connections originate from somewhere within their own network. That means it would be impossible to directly connect to the ISP mail server from some device outside the subnet controlled by the ISP, say a smartphone.
- the second reason is that we want to be able to send local mails originating in our server (for example system mail) to a valid address (this will also require us to alias a local user to a real email address since most mail servers refuses to pass on messages with a bogus sender address, say "<root@localhost>")

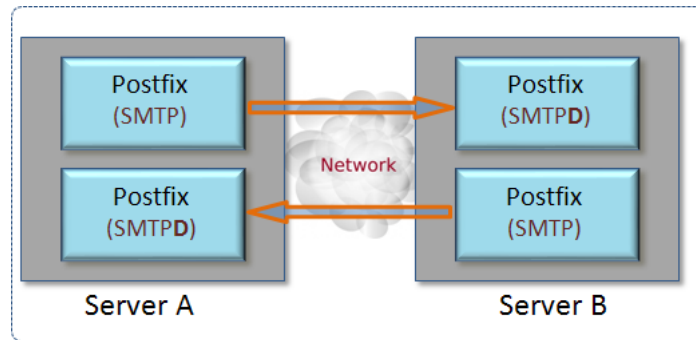


Figure 3: The dual role of postfix

2.2 Dovecot

All messages need to be stored in a structured way on disk for each user. The two formats mostly used in the Unix/Linux world are.

- "maildir" format. This format stores all messages as individual files in a directory structure corresponding to the mail folders defined.
- "mbox" format. This format stores all messages lumped together in one file. Historically this is the "original" message storing format. For all practical purposes this should not be used in new setups.

For all modern installation it is strongly recommended to use the "maildir" format. This is the format we will setup Dovecot to use in Section 5, "Setting up Dovecot".

In order to access the mail storage the most flexible way is through an IMAP server (the old and mainly obsolete way being through a POP3 server). It is important to understand that an IMAP server is only responsible to manipulate messages (files) on disk storage. The IMAP server is like mother goose watching over the mail storage. It is not involved in neither sending nor receiving messages in any shape or form. The mail user agent (MUA) connects to the IMAP server and reads the messages for its user. Messages are always kept at the server even after they have been read. This will allow us to later synchronize the message storage with any other devices and messages that are marked as read on one device will show up read on all other devices as soon as they synchronize their mails. For efficiency reason most mail clients choose to cache messages locally in order not to have to transfer them from the server each time the user decides to open the message. In our case we will later on show how to setup a web based mailer and in that case there will be no local caching and messages are only stored on the server.

For now we can also assume that it is the responsibility of some other program to actually store the incoming messages in the mail storage (on disk). In mail lingo such a program that stores messages locally on disk is called LDA (*Local Delivery Agent*).

The main responsibility for an IMAP server is to allow authenticated users to access their messages and update the status of messages such as read or moved to a different mail folder.

So how can Dovecot keep track of mails if any other program can write new mails into the storage?

On Linux Dovecot uses notifications of file changes to get information on when new mails arrive (using the inotify mechanism). By subscribing to file changes in the mail directories it is possible for Dovecot to maintain information and indexes (to quickly find mail). There is also a structure in the maildir directories on where to store new mails so Dovecot knows exactly where to look for new messages.

One reason for choosing Dovecot as our IMAP server (there are others as well) is the fact that it is fast, extremely robust and it is almost impossible to get into a corrupt IMAP state. Even removing all the index files will have no effect. Dovecot will just notice that the files are gone and recreate them by re-reading mails in the storage as necessary.

Dovecot is also the only widely available IMAP server (including some well-known commercially available servers) that fully passes the complete IMAP stress test (see <http://www.imapwiki.org/ImapTest/ServerStatus> [<http://www.imapwiki.org/ImapTest/ServerStatus>])



Note

Dovecot is to large extent the creation of a single individual, a finish software engineer with the name of *Timo Sirainen*.

More information on Dovecot can be found at <http://dovecot.org/>

2.3 Fetchmail

Since external MTA servers can not connect to us directly in order to periodically push new messages to us we must have some method to gather messages from external mail accounts and store them in our mail repository. This is the only way we can receive messages assuming we are configuring a home server which lacks the ability to act as an official domain mail server. Enter here the aptly named program fetchmail.

This program is used to periodically authenticate and fetch new messages from external mail accounts and deliver them locally. We can, for example, setup fetchmail to gather messages from a GMail or Hotmail account so that they can later be accessed by the IMAP server.

The way fetchmail works is that it collects messages from external accounts and then delivers them to our MTA (postfix) on port 25. In effect making it look to postfix like it has received a mail from another MTA, see Figure 4, “fetchmail process overview”.



Caution

Since fetchmail does not handle authentication at delivery on port 25 postfix must be setup to accept delivery from other daemons that run on the same server without requiring authentication. We will later show how to do that in the postfix setup section.

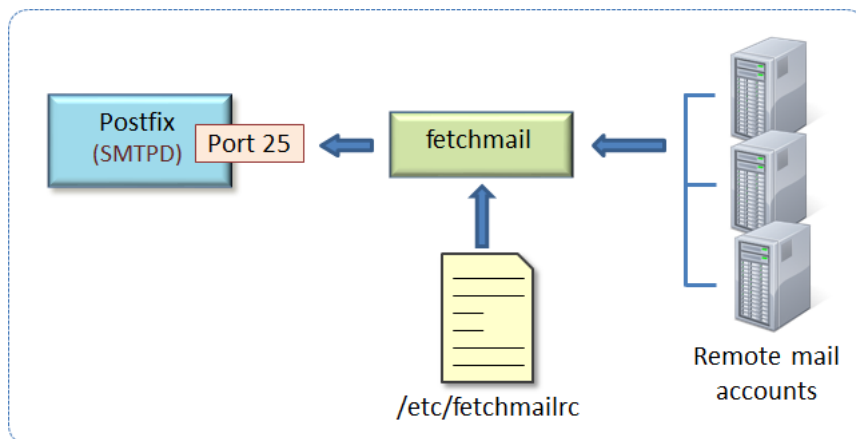


Figure 4: fetchmail process overview

Refer to Figure 2, “Mail server overview” for an illustration where fetchmail fits into the overall setup.

2.4 Procmail

The final bit of the puzzle, which strictly speaking is not necessary, will make things like spam filtering and rule based processing of messages possible. Procmail fills the role of a LDA (Local Delivery Agent), the last stage that receives a mail (from postfix) as input before storing it on the file system mail storage.

Procmail configuration is based on rules which uses pattern matching of messages together with a corresponding action to be taken when the pattern matches.

Depending on the user specified rules different actions can be taken depending on the content of the message. A common rule is to store incoming messages in a specific folder based on some criteria. This could for example be used to store identified spams in a specific folder, newsletters in another and the rest of the incoming messages in the inbox. The action in the rules are not just limited to sorting mails in different folders, external programs can also be called when a pattern matches.

The rules that control procmail on how to handle incoming mails are very flexible and it is easy to do things like passing certain mails to some programs for further processing or make automatic replies based on the content in the mail.

Unfortunately the syntax for specifying rules in procmail is an acquired taste at best and takes some getting used to. To ease the writing pain we have collected a few common and useful procmail recipes in Appendix B, *Procmail recipes*. On the net there are also dedicated pages with recipes for several commonly used procmail rules.

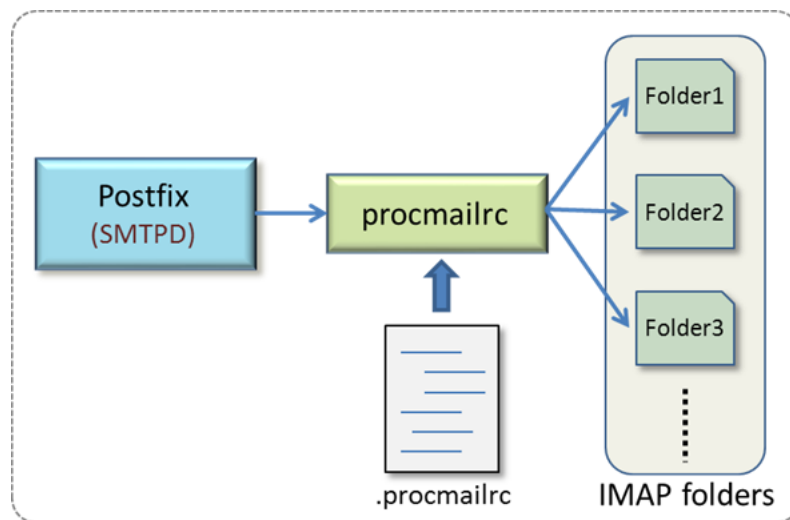


Figure 5: Interaction between Postfix and procmail

2.5 Roundcube (and Apache)

The above four actors (postfix, dovecot, fetchmail and procmail) are only one possible set of applications that can be used to create a basic mail server. However this particular combination can probably be considered one of the standards that both individuals larger organizations use.

In our setup we will also install a Web based mail client that will allow reading and manipulate of messages using any available Web browser.

The mail client will be installed in the same server and talk directly to our IMAP server, i.e. Dovecot. In our setup we will configure Apache (which is our web-server) to allow `https://` protocol access to Roundcube.

We have chosen Roundcube since in our opinion it is one of the best free Ajax-based web-mail clients available and is easy to install. A screen shot from Roundcube is shown in Figure 6, “Roundcube Webmail client”



Note

If only native clients should be used (for example Thunderbird or KMail) skip this step and use a native mail client (for example KMail in the KDE desktop) as a mail client and configure it to access the local IMAP mail storage.

However we would still recommend to install Roundcube since it is straight forward and gives universal access to the mail storage from anywhere in the world as long as a browser with internet connections can be found.

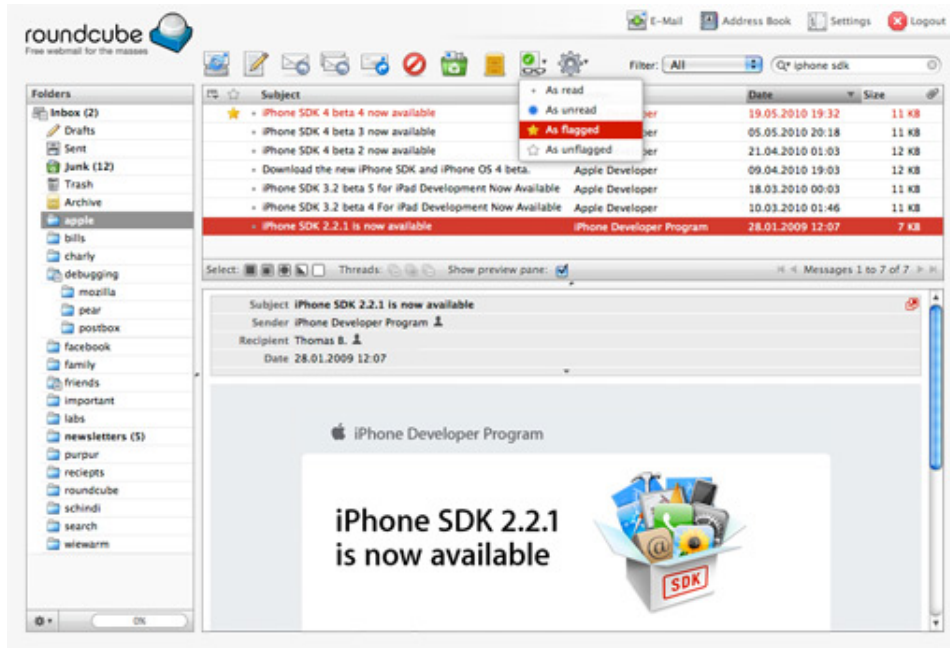


Figure 6: Roundcube Webmail client

2.6 openssl

This is a supporting library used by several other components. The openssl library is the de-facto standard encryption library on Linux based systems. It is used to both manipulate and create encryption keys as well as being library for other programs needing to encrypt data. The term openssl can refer both to the specific command **openssl** as well as to the overall library, the context will determine the suitable interpretation.

This library will among other things be used to setup a secure connection to our Webmail based on the HTTP protocol. We should already now say that the management of SSL (Secure Socket Layer) connections and the encryption/decryption keys via this library and its tools is somewhat convoluted. A large part of the confusion (as witnessed by the countless requests for help on the net) with this library is that very few takes the time to understand it. As a result many of the "recepies" found on the net is obsolete, misuses standards or are plain wrong. Unfortunate the developers of openssl and its tools have been kind enough to maintain compatibility with very old ways of doing things. This results in that old recepies that uses very antiquated ways still (kind of) work. Since most people (even administrators) are more than happy to just use something that seems to work (without understanding it) this has reulted in both some very bad habits when using openssl as well as potential security weaknesses and inconveniences which could be easily avoided.

To remedy this situation we will explain in detailes and show the modern and secure way of setting up the necessary encryption/decryption infrastructure. To really cover all the details in an accessible way we have also added an entire appendix describing how to use the library Appendix E, *Create SSL/TLS keys and certificates with openssl*.

2.7 spamassassin

To help identify spam mails (i.e. unwanted junk mail) we will setup and use a Perl library called spamassassin. When a message is about to be delivered to the local user it will first be run through spamassassin. This program will then examine the message based on a number of both pre-defined and heuristic tests and give the mail a score to determine the likelihood that the message is junk. The calculated score is added as an additional header in the message.

After the score has been determined it is entirely up to the user that received the message to use the calculated score for further actions. The most common example is to have all messages above a certain score to be stored

in the junk mail folder and those with a really high score to be deleted right away. We will later on show how to add procmail rules to handle this.

3 The setup process

It is now time to start the setup process. There are a number of steps that needs to be done and as we progress each step will be tested separately as far as possible before we move on to the next and finally the complete system. The steps are explained below and also illustrated in Figure 7, “Steps in the setup process”:

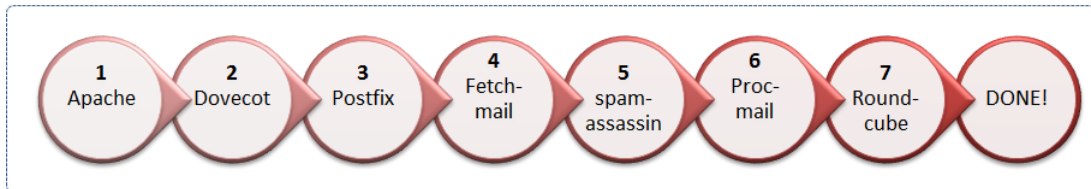


Figure 7: Steps in the setup process

1. Setup Apache to accept **https://** traffic by creating a self-signed certificate for Apache. This is necessary so that we can access Roundcube (see below) securely. We start here since this is a self-contained task and can be tested separately.
2. Setup Dovecot as an IMAP server. We will also configure the server to use a secure connection based on SSL/TLS so we can securely access our mails. This requires us to create another set of self-signed certificates to be used by Dovecot. In addition we will also configure Dovecot to be used as the authentications server for Postfix so that only authenticated users can send mail through our SMTP server (i.e. postfix) this is **very important** to avoid creating an open relay that could potentially be exploited by a spammer.
3. Setup Postfix to both accept incoming SMTP traffic (with authentication via Dovecot) as well as sending mail as a relay through to the ISPs SMTP server if the recipient is non-local. We also need to instruct postfix to use procmail as LDA ([Definition: Local Delivery Agent]) to store the mails in a suitable location in the mail storage.
4. Setup fetchmail to gather mail from any external mail providers and deliver it back to postfix.
5. Setup spamassassin as a spam filtering engine in the background by using its daemon mode option. Received mails are then run through the spamassassin client (**spamc**) by procmail.
6. Setup procmail to use spamassassin to filter spam and store them in a separate folder on the IMAP server. The emails will be delivered to procmail from Postfix (piped) when postfix has decided that the mails should be delivered to a local user. Postfix will invoke procmail to do the local delivery as the user who's mail we are going to deliver. The instructions to procmail will then be read from <user home directory>/**.procmailrc**.
7. Finally we will install and setup Roundcube to access the IMAP server to be able to read and manipulate mails via a Web-interface. This will also include the configuration of a database (MySQL) for Roundcube to make use of.

3.1 Some ports and acronyms used

For the TCP/IP communication there are a number of ports and acronyms that are used in a mail setup and it is good to have a shortlist handy. Table 2, “Port numbers relating to mail handling” and Table 3, “Acronyms used in this tutorial” list the ports and terms we will be using. For historic reasons several port numbers have been used for the same purpose and can lead to some confusion. This is especially true for the port numbers possible to use for secure SMTP. This is further elaborated in the section dealing with procmail setup.

In our setup we will use the secure version of all the protocols involved. This will unfortunately add some steps in the process but doing anything else would be irresponsible!

Table 2: Port numbers relating to mail handling

Port	Protocol	Comment
80	HTTP://	Hypertext protocol
443	HTTPS://	Encrypted hypertext protocol over SSL/TLS
143	IMAP	[Definition: Internet Message Access Protocol] used to access a mail storage
993	IMAPS	Encrypted IMAP protocol over SSL/TLS
25	SMTP	[Definition: Simple Mail Transfer Protocol] used to speak with a mail server
465	(SMTPS)	Simple Mail Transfer Protocol with SSL. Unfortunately this is not an officially recognized/registered port for this purpose. Historically this can be traced back to Appendix A of the 1996 non-standard standard The SSL Protocol Version 3.0 [http://wp.netscape.com/eng/ssl3/draft302.txt] This port should not be used in a modern server setup. Use port 587 (see below) instead.
587	SMTP/SMTPS	Email submission (See RFC 2476). This is the officially registered port that should be used when a mail client submits a message to a mail server. The difference with Port 25 is that port 25 should only be used for mail-server to mail-server communication. For historical reason, port 25 has also been used to accept mail client submissions.
110	POP3	[Definition: Post Office Protocol]. The "old" way of accessing a mail storage. Should not be used any more (only listed here for completeness)
995	POP3S	Encrypted POP3 protocol over SSL/TLS

Table 3: Acronyms used in this tutorial

Acronym	Description
SMTP	[Definition: Simple Mail Transfer Protocol], protocol used to send mail between servers. Note do not confuse with the name of the postfix two daemons SMTP/SMTPD
SMTPS	Encrypted SMTP protocol (using TSL/SSL)
IMAP	[Definition: Internet Message Access Protocol], The protocol between a mail client (e.g. Roundcube) and the mail server. Allows manipulation of mails and synchronizing local mails to a local storage.
IMAPS	Encrypted IMAP protocol (using TSL/SSL)
TLS	[Definition: Transport Layer Security] (formerly known as SSL=[Definition: Secure Socket Layer]). There are technical differences but for end users they do behave the same. There is however one crucial difference in that SSL uses encrypted communication from the very start while TLS uses plain text initially and once the two communicating partners has agreed on encryption the remaining communication is encrypted. This has the advantage that the same port can be used for both encrypted and non encrypted version of a protocol.
MTA	[Definition: Mail Transfer Agent], generic name for the actual program that speaks SMTP (in our case postfix)
MUA	[Definition: Mail User Agent], generic name for the actual mail client (Roundcube in our case)
LDA	[Definition: Local Delivery Agent], generic name for whatever program is responsible for the final mail delivery to the recipients mail box (procmail in our case)
PAM	[Definition: Pluggable Authentication Modules]. A generic framework for authentication in a Linux system. This allows clients to perform authentication on a system without knowing the exact methods this authentication is done by. The historic way of performing authentication is

Acronym	Description
	by the usage of a user/password file but other ways are available as well, for example via LDAP ([Definition: Lightweight Directory Access Protocol]) databases.
SASL	[Definition: Simple Authentication and Security Layer]. This is another generic framework for authentication but intended to be used by various internet protocols. Usage of this framework allows decoupling of protocol authentication from the actual system it is talking to.
PEM	[Definition: Privacy Enhance Mail], A file format to store SSL/TLS key pairs, commonly used in Unix like systems.

3.2 Using SSL/TLS certificates

It is impossible to avoid SSL/TLS ([Definition: Secure Socket Layer] / Transport Layer Security) certificates when configuring a mail server using encrypted communication. We must therefore start with a simplified explanation of key pairs and certificates since they are the heart in the encryption of the data between a client and a server. We will use openssl to manage the so called SSL/TLS encryption (this is used by most people in the Unix world). In Appendix E, *Create SSL/TLS keys and certificates with openssl*, we discuss more in detail how to use this library and associated programs directly to create the necessary keys.

The openssl library supports most (if not all) public used cryptos intended for secure communication and has a very large number of commands and sub-commands. In the following we will only shortly discuss the basic usage of openssl since a full discussion is outside the scope of this tutorial.



Caution

openssl has been around for a long time and as it has evolved old parameters which for all practical purposes and intents are deprecated still remains in the program to ensure backward compatibility. This means that the number of commands, options and suboptions possible to give to openssl is mind boggling. Consequently a number of scripts available on "the net" uses old deprecated methods since there are very little incentive to learn new (and better) ways of doing things. The learning curve for openssl is quite steep but luckily we only need a small subset of available commands.

Certificates and keys

In short the SSL/TLS certificates (or SSL/TLS key-pair) serve two purposes:

1. Form the encryption base for the public key anti-symmetric encryption (different keys for encryption and decryption) used in the SSL/TLS (openssl can also handles symmetric encryption).
2. Prove the identity of the server that you are connecting to

Public key encryption is based on two different (anti-symmetric) keys, one public key and one private key.

The common way is to use the public key to encrypt data to be sent and the private key to decrypt the same data. However, since the encryption is anti-symmetric it can also be used the other way around. Encrypting with the private key and decrypted with the public key. This can be used to assure that a message comes from a trusted sender as follows.

Assuming we know with 100% certainty that the public key we have really comes from a specific server. If the server then encrypts a message string with its own private key we can decrypt it with the public key and if it decrypts correctly to a valid message we can be reasonable sure that the message really comes from that specific server.



Note

When talking about SSL/TLS certificates one should really refer to them as X.509 certificates which was standardized a long time ago by ITU-T. The original designer of SSL/TLS certificate handling, Netscape (anyone remembers them?), used the already existing standard of X.509 certificates when they created the SSL/TLS protocol. The use of X.509/SSL/TLS certificates on the internet is described in RFC 5280. The name SSL/TLS certificates has stuck (probably since it is easier to say than X.509) and we will be using this name in this text. However when using `openssl` to create certificates one specifies X.509 as the format for the certificates so it might be good to know and understand where this strange name comes from.

The public key in this context is usually referred to as a certificate and is by convention often stored in a file with the ending “`*.crt`” and the private key (again by convention) is stored in a file with the ending “`*.key`”. This naming convention is unfortunate since the keys are actually stored in a format known as PEM (see the fact box below).



Note

The most common format in the Unix world to store the certificates in is in the so called PEM (originally from [Definition: Privacy Enhanced Mail]) format. This is basically a base64 encoding of the binary data that makes up the key (the X.509 specifies the binary format to be in ASN.1). The base64 encoding makes it possible to treat the file as ordinary text files. There are other possible binary formats of keys (and certificates) Windows usually use one of those but we will not discuss this further here.

A common practice to differentiate the contents of the files where the public and private keys are stored is to use different suffixes on the certificate and the private key files.

Even though the keys are in the same format (PEM) they are stored in files with different suffixes. One very common way is to give the public key the ending “`*.crt`” and the private key the ending “`*.key`”. This is not ideal since our view is that the file suffix should indicate the technical format/encoding of the file. However, this is common practice so we just accept this for now. (In Appendix E, *Create SSL/TLS keys and certificates with openssl* we show another naming standard that makes more sense to us.)

It is also possible to store both the public and private key in the same file. This is useful for applications where we are never expected to manually hand out a public key but where this is done programmatically between programs/system. The advantage of storing both the public and private key in the same file is obvious; one never risks of mixing the public/private key with the wrong other key.

The PEM format specifies a textual header before the private and public key so they are easily identified even if they are stored in the same file.

In order to prove the authenticity of a public key some companies that are trusted (a [Definition: certification agency], CA) will first encrypt the servers’ public key (against a quite hefty fee). When the public key is distributed it can be verified by the client by decrypting it with the CAs public key. The public key from the CA has to have been previously distributed in some other way so that the client has access to that public key in safe way. Therefore such a public key, which is usually a root certificate, is included with the distribution of browsers. The principle of this is shown in Figure Figure 8, “SSL/TLS Certificate signing process”.

Certificates can also be chained. A super trusted company can sign a certificate from a trusted company which in turn could sign your certificate (an intermediate agent). If the browser has the (root) certificate from the super trusted company it will implicitly also validate any child certificates stemming from this super trusted company.

In our setup we will become our own certification agency and create a “super” key pair that we will use to sign (encrypt) all the public keys we will be using. Hence the name self-signed certificates. Technically speaking this is only one definition of the term self-signed certificate and is theoretically not quite correct. It is however a common “miss-use” of this term. In Appendix E, *Create SSL/TLS keys and certificates with openssl* we elaborate on this.

Protocols using encryption in our setup

In our setup we will use SSL/TLS certificates (encryption) to handle the following protocols:

- HTTPS - Encrypted communication to our Web-server
- SMTPS - Encrypted communication with our Mail-server (postfix)
- IMAPS - Encrypted communication with our Mail-storage-server (Dovecot)

In principle we could use the same key-pair for all these channels but in our setup we will use one key-pair for the Web-server and one key-pair for both the SMTPS and the IMAPS channels. The motivation for this is that the Web-server is one standalone server which could be used for a multitude of applications and the SMTPS/IMAPS channels logically belong together as one application. The public HTTPS key should also be distributed to Web-clients wanting to connect to our web-server without getting browser warnings.

In addition we will create our own CA (root) certificate that we will use to sign all these key-pairs (certificates). Our hierarchy of certificates is shown in Figure 9, “Certificate hierarchy”. The public root certificate should also be distributed to all clients for them to verify the other certificates.

We should also make a note of the key length here. By default openssl uses a key lengths of 1024 bits. To protect ordinary mail from the occasional snooping this is good enough.

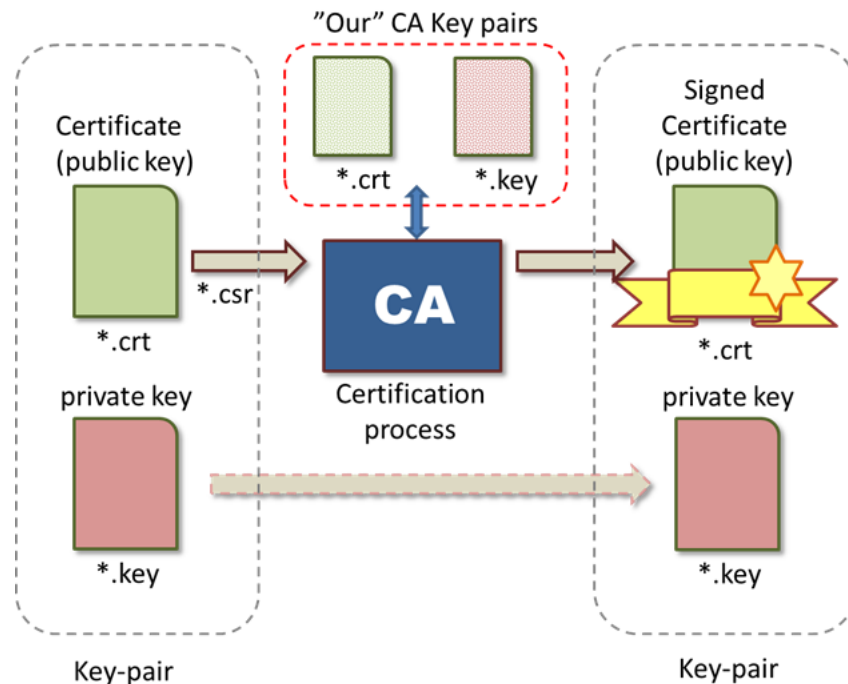


Figure 8: SSL/TLS Certificate signing process

However, for corporate level protection a key length of at least 2048 bits is recommended and for even further protection a key of 4096 bits can be used. The main drawback of longer key lengths is the increased processing power and time it requires for decryption/encryption.



Note

From 2011 Microsoft has removed all 1024 bit root certificate from the distribution of its browser.

A valid question now is who signs the CA (root) certificate?

The answer is no-one. These certificates (key-pairs) are trusted because we say so (they are truly self-signed). They don't need higher authority, technically they have a flag set in the certificate saying that *"this is a trusted root certificate that doesn't need anyone else to vouch for it"*, hence we should trust it. This also means that when we install these root certificates we have to make sure we are installing them from a secure source that we trust. It could potentially be devastating if we were to install bogus root certificates that were controlled by an "evil" entity since they could potentially fool us into believing that a signed program was produced by a trusted party when it in fact could be a spy program.

Browsers, for example, comes with a large set of root certificates from trusted sources as do all Linux distribution. In the case of OpenSuSE all trusted root certificates can be found under the directory `/etc/ssl/certs/`

All key-pairs can be created by using the `openssl` package (and more specifically the `"openssl"` command with various options). As mentioned before the steps to create these keys (certificates) and doing the signing are not really difficult taken step by step (and is shown in Appendix E, *Create SSL/TLS keys and certificates with openssl.*) but there are a number of steps to be taken and since even professional system administrators do this fairly seldom there is no point in committing all the detailed steps to memory.

Exactly for this reason there are a number of shell scripts floating around that lines up the commands needed to create all these certificates so one does not have to memorize all the steps in using the `openssl` library. The library and associated tools are not exactly user friendly so even professionals that deals with this library on a regular basis create there own utility script to make life easier. Unfortunately there isn't a set of "commonly agreed" utility scripts.

X.509/SSL/TLS certificates introduces several terms that must be understood. The most important is the [Definition: Distinguished Name] or DN. This is a unique name used to identify a particular certificate (and key) among all other 1000s of certificates in the world. Hence this must be unique to each certificate.

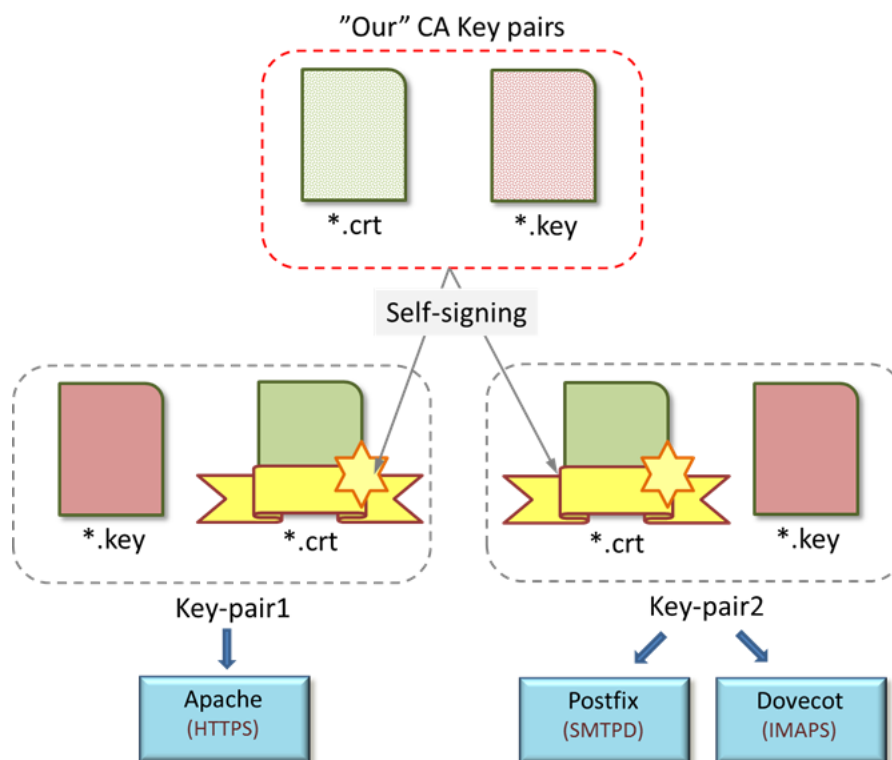


Figure 9: Certificate hierarchy



Note

Beware that many SSL/TLS scripts floating around on the net are "Chinese whispers". While they do work they use since long obsoleted options and ways to do things that are no longer recommended (at least since

2004!). The reason is most likely that getting into the details of openssl is not something many people want to do. So finding a small script on the net that seems to do the job is good enough for most people and so the scripts (unfortunately) gets handed down to the next unsuspecting generation.

Again, in Appendix E, *Create SSL/TLS keys and certificates with openssl* we show the proper (and modern) way of handling openssl.

The DN is in turn made up of several fields such as organization name, unit within the organization, town, state etc. One field in the DN with very special meaning is the [Definition: Common Name] (or CN) field. This is used to identify the domain name (e.g. `www.example.com`) that is used to access the server where the certificate is valid. If the CN differs from the URL that is used to access (for example) a Web-server you will normally get a warning that the authenticity of the server cannot be guaranteed. The CN is normally the fully qualified domain name (FQD) of the server. This poses some problems if the server is multi-homed, i.e. it can be accessed using different URLs, for example if both `http://www.example.com` and `http://example.com` are valid aliases. For this reasons there are extensions (in the X.509 standard) that makes it possible to add a number of aliases that the certificate should be valid under (all aliases must however still resolve to the same IP address). How this is done is further described in Appendix E, *Create SSL/TLS keys and certificates with openssl*. This is also an example of quite important information that is not commonly found on all the sites that discuss certificate handling.

The final step after creating all the certificates is to install the public CA (the root certificate) that was used to sign the certificates in our browser and possible mail client(s) to avoid warnings about untrusted certificates. This is strictly speaking not necessary but will avoid annoying warnings whenever a connection is made to the server, for example via a Web-browser.



Note

A X.509/SSL/TSL certificate is not exactly identical to the public key. A certificate acts as a container for the public key and adds additional information such as who has issued the certificate and who has guaranteed the authenticity of the certificate and various other information such as how long the certificate is valid for.

If more information about a certificate is wanted openssl can display a more detailed textual printout. The following examples shows how to printout information in various situations.

To view information on the public certificate use the command

```
$> openssl x509 -noout -text -in mycert.pem
```

To view the technical details about the private key use the command

```
$> openssl rsa -noout -text -in mykey.pem
```

To view the details about a certificate request use the command

```
$> openssl req -noout -text -in mykey.pem
```

In the following description of the setup of Apache and Dovecot/postfix we have made use of the utility scripts that are available with OpenSuSE since this is the easiest way to get up and running without having to learn too many details about openssl. However, after a working setup is achieved we strongly encourage reading through Appendix E, *Create SSL/TLS keys and certificates with openssl*, where a more satisfying and complete description and setup of SSL/TLS certificates can be found. Reading through that appendix will take away part of the “black-magic” clouding this area.

4 Setting up Apache HTTP-server

To setup Apache we will first create and enable SSL/TLS certificates to be able to connect over `https://` protocol to the server. After this we will define a separate document root for the HTTPS traffic and finally enable PHP5 for the secured version of Apache.

To separate the normal document root (that is accessed through `http://`) and the secure root (accessed through `https://`) we will also create a new secure document root at `"/srv/www/ssl-htdocs"`.

In the following we will assume that you have installed Apache v2.x together with the Apache PHP5 module through SuSE package management.



Caution

Please make sure that you can access a test page using normal HTTP traffic to your Web-server before continuing!

4.1 Creating self-signed SSL/TLS certificate for Apache

Creating SSL/TLS certificates for the `https://` protocol as is mentioned above is straightforward but tedious. The good news here is that in OpenSuSE there is a shell script that does all the steps involved in creating certificates semi-automatically using `openssl`, **"gensslcert"**. The built-in help in this script will show all necessary options for the certificates. So running

```
$> gensslcert -h
```

will give you the list of all options. In the example command line below we will assume that the server resides in Sweden (country code SE) and accessed through the IP address `192.168.0.50` (in the address range reserved for a local net which is not directly connected to the internet). The IP address specified on the command line is rather important (it is the CN name) as described in the previous section on SSL/TLS. When your browser accesses the server it will check that the certificate it received from the server matches the URL used to access the server. If there is a mismatch the browser will generate a warning that the site should not be trusted (since the certificate could have been made out to another server and then just copied). This might cause some irritation since it is common for a home-server to be multi-homed, i.e. accessed via different paths or URLs. An example might be a dynamic DNS setup to access the home server from anywhere in the world with a domain name of choice, say `"http://myserver.dyndns.com"`. In addition a server can also be accessed in the local network, for example as `http://192.168.0.50`.

So there is a choice to be made here. The domain name which you most often use to connect to your server should be used in the certificate (as given in the Common Name field). This way you get the fewest warnings when connecting to your server.



Tip

With a more advanced setup for the certificate handling, described in Appendix E, *Create SSL/TLS keys and certificates with openssl*, it is possible to setup several "alias" addresses for the same server (using the AltN field in the certificate) to avoid this kind of irritating warning messages.

in our case we will use the IP as the common name for the server certificate so we call the utility script as:

```
$> gensslcert -y 3650 -Y 7000 -c SE -n 192.168.0.50
```

This will create both our own trusted root certificate as well as the certificate for the server itself and then sign the certificates with our own root certificate. Our self-signed certificate will be valid for 10 years (~3650 days) and valid for the server that is accessed by the IP address 192.168.0.50. Our CA (root) certificate will be valid for roughly 20 years (~7000 days). It is still possible to access the server through other URLs for example by its name (if you have put it in your “/etc/hosts” file or even have a local DNS setup) but the browser will issue a warning in that case that it cannot validate the authenticity of the server. The exact wording of the warning (and appearance) is browser dependent.

After you have run this script the necessary certificate files will be stored under various subdirectories “/etc/apache2/ssl.*” as shown in Table 4, “Certificate/keys generated by “gensslcert””.

After the certificates have been created we must now enable SSL and set the document root correctly in Apache for the virtual host which will handle the incoming HTTPS connections.

Table 4: Certificate/keys generated by “gensslcert”

Cert/key file	Description
ssl.crt/192.168.0.50-server.crt	X509 SSL/TLS certificate. The certificate contains the public key of the server.
ssl.key/192.168.0.50-server.key	SSL/TLS private key. Must be kept private at all times!
ssl.crt/192.168.0.50-ca.crt	The CA root certificate (public key). This file should be imported to your Web-browser to avoid the “unrecognized certificate” warning when you are connecting to your HTTPS connection.
ssl.key/192.168.0.50-ca.key	The CA root private key. Should be kept private at all times!

4.2 Enabling SSL in the OpenSuSE configuration

Many of the configurations in the apache configuration script templates available with OpenSuSE have conditional statements whether SSL/TLS is enabled or not. This is controlled by a single flag stored in the configuration file “/etc/sysconfig/apache2”. In this file a number of variables define the setup for Apache. Here there are two variables that must be correctly set.

First the string variable `APACHE_MODULES` has to have “ssl” defined in the list of modules to be loaded. This module is added by Yast when Apache is installed. If this module is missing there is a risk that you have not properly installed Apache with the necessary `mod_ssl` module.

Secondly the SSL flag must be set as a Apache server flag as

```
APACHE_SERVER_FLAGS="SSL"
```

This must be set in order to enable the lines surrounded by a conditional “`<IfDefine SSL>`” statement in the various Apache configuration files.



Note

To avoid manually editing `/etc/sysconfig/apache2` in order to enable or disable Apache modules or flags there are two command line tools that can do this, “`a2enflag`” and “`a2enmod`”. These two commands are used to disable/enable flags and modules in `syconfig` file.

For example the following two commands would add both the SSL flag and the module `mod_ssl` to the configuration file

```
$> a2enflag SSL
$> a2enmod ssl
```


To list all the currently enabled modules use the following command (the modules listed below are just examples and does not necessary reflect what must be loaded.)

```
$> a2enmod -l
actions alias auth_basic authn_file authz_host authz_groupfile \
authz_default authz_user autoindex cgi dir env expires proxy \
proxy_http proxy_html include log_config mime negotiation \
setenvif ssl userdir rewrite php5 dav dav_svn
```

4.3 Setting up a virtual host for HTTPS

What we want to do now is to make use of the certificates we created and have Apache look in the alternative document root “/srv/www/ssl-htdocs” when the server receives a request over “https://”



Note

Strictly speaking there is no need to have a separate document root for HTTPS. It is possible to use the same document root as for normal HTTP traffic. However, for security reasons it is better to put pages that should only be accessible through the HTTPS in a separate root to avoid the risk that they are accessed through the normal HTTP protocol by mistake.

We will be very short in the description here since configuring Apache is not necessarily difficult but requires a lot of text to explain all the details. So in this context we will just show what is needed with a minimum of explanations (there are entire books written on Apache configurations to consult). In Appendix F, *Apache virtual host example configuration*, there is also a complete configuration file as an example.

To setup the new document root for the https:// connection start with the following four steps:

1. Switch to root user
2. Create the new document root for HTTPS://

```
$> mkdir /srv/www/ssl-htdocs
```

3. Change to directory /etc/apache2/vhosts.d
4. Copy the template config file for an SSL-enabled server as a starting point

```
$> cd /etc/apache2/vhost
$> cp vhost-ssl.template vhost-ssl.conf
```

We shall now edit the `vhost-ssl.conf` file so that Apache finds the certificates we want to use and the new document root we just created. Find the following variables in the configuration file we just copied (`vhost-ssl.conf`) and change them as shown below

```
# Setup document root and server name
DocumentRoot "/srv/www/ssl-htdocs"
ServerName 192.168.0.50:443

# Setup log files
ErrorLog /var/log/apache2/ssl-error.log
TransferLog /var/log/apache2/ssl-access.log

# Tell Apache where to find the certificate files
SSLCertificateFile /etc/apache2/ssl.crt/192.168.0.50-server.crt
SSLCertificateKeyFile /etc/apache2/ssl.key/192.168.0.50-server.key
```

```
SSLCertificateChainFile /etc/apache2/ssl.crt/192.168.0.50-ca.crt
```

After you have changed this we need to tell Apache to accept the new document root. Add the following lines just after the `TransferLog` directive (as long as it appears within the `<VirtualHost _default_:443> ... </VirtualHost>`) it is really irrelevant where you put it)

```
<Directory "/srv/www/ssl-htdocs">
  Options None
  AllowOverride All
  Order allow,deny
  Allow from all
</Directory>
```

To also enable PHP5 and any other configurations that is available for the standard Web-server we include all the other configurations files as well by adding the line below Just before the `<Directory>` directive (this assumes that you have installed `mod_php5` for use with Apache previously)

```
Include /etc/apache2/conf.d/*.conf
```

Any additional configuration files added to your Apache setup (possible by Yast) will be read also for our encrypted setup in this way.

Please look at the (very) readable Apache documentation for a more detailed explanation of possible setups. It is also highly likely that in the end you may want to tailor the setup according to your own needs (most likely you want to change the `Options` parameter for the document root directory, for example to allow symbolic links to be followed).



Tip

In Appendix F, *Apache virtual host example configuration* we show an example of a complete `vhost-ssl.conf` file.

4.4 Restarting and enabling Apache

What remains now is to restart Apache and make sure there are no errors. To do this execute the following commands as root and verify that there are no errors. If there are any errors go back and check for spelling mistakes in the configuration files. Error messages to help pinpoint the problem can usually be found in `/var/log/messages` or in the directory `/var/log/apache2/`

```
$> rcapache2 restart
Syntax OK
Shutting down httpd2 (waiting for children to terminate)    done
Starting httpd2 (prefork)                                   done
```



Tip

If you are used to SysV ini files you could as well start the server with the line

```
$> /etc/init.d/apache2 restart
```

`/sbin/rcapache2` is just a symbolic link to the corresponding `init.d` script

After this you probably also want to have Apache start automatically each time you start your server. If you haven't already done so give the command (as root):

```
$> chkconfig --add apache2
```

This will add Apache daemon as a service in the default run levels for your machine. This could also be done by opening Yast and go into <system->run levels> and add Apache as a service to be started.

Before continue verify that you can make a connection to Apache over HTTPS (for example by just creating a single line “index.html” in the root directory and just call `https://localhost/index.html` from the browser)

Be aware that there will be a warning about this being an untrusted site since the root certificate we use have not been installed in the browser as a trusted certificate. For now just ignore this.

5 Setting up Dovecot

The reason to start with Dovecot rather than Postfix is twofold. One is that postfix will make use of the Dovecot SSL/TLS certificates (as shown in Figure 9, “Certificate hierarchy”) and the second is that postfix will use Dovecot SASL service to authenticate incoming SMTP requests. SASL stands for Simple Authentication and Security Layer and is a standardized protocol to handle authentication. The reason for this is that we want incoming SMTP traffic to be authenticated forcing users to log in before being able to send mails through our SMTP(s) server. Since postfix does not provide any authentication service by itself it must make use of some service provided by another component, Dovecot in this case.

The reason for re-using the SSL/TLS certificates from Dovecot in postfix is partly laziness; Dovecot comes with a small script that makes the creation of certificates very easy.



Tip

We also remind of the more complete description on how to use openssl directly in . to create a certificate hierarchy with a bit more control that comes from better understanding. But for now we make use of the utility script that comes with Dovecot2 since it is the easier way for a newcomer to do what needs to be done.

In the following we assume that you have installed Dovecot2 through Yast. Before continue please make sure that Dovecot 2.0.x is installed and not 1.x. For some reason SuSE still uses 1.x as the default version of Dovecot (instead of Dovecot2).



Note

The script that comes with Dovecot does roughly the same step as the previously used **gensslcert** (for Apache2) but stores the resulting certificate files in a different directory and also uses the *.pem endings instead of *.crt/*.key (see the section on SSL/TLS certificates for an explanation of *.pem vs. *.crt/*.key suffixes)

5.1 Modifying dovecot configuration files

After installing Dovecot all configuration files can be found under `/etc/dovecot/conf.d`

The files that need editing are listed in Table 5, “Dovecot configuration files that needs modifying” together with a short explanation of the purpose of each file.

Table 5: Dovecot configuration files that needs modifying

Configuration file	Comment
<code>10-master.conf</code>	Enabling and disabling what protocols Dovecot should listen for
<code>10-auth.conf</code>	What methods to use for user authentication
<code>10-mail.conf</code>	Location for mail repositories
<code>10-ssl.conf</code>	Settings for encrypted communication
<code>10-logging.conf</code>	Settings for logging from Dovecot. Useful when debugging issues.

Strictly speaking there is no need to change the last file controlling the logging levels but while setting up the server it is very handy to enable as much logging as possible in order to ease any troubleshooting that may be needed if things don't quite work out as we think.

10-master.conf

Here we need to enable only IMAPS. Make sure that the section service IMAP-login has the following settings:

```
service imap-login {  
  inet_listener imaps {  
    port = 993  
    SSL = yes  
  }  
}
```

Further down in `10-master.conf` the configuration for the authentication socket can be found. This will later be used by postfix to check any incoming logins. By default only root is allowed to read on this socket so as long as postfix is run with root privileges the default permissions are fine. However normally postfix is run as a separate user for safety so these permission needs to be relaxed as shown below.

```
service auth {  
  unix_listener /var/spool/postfix/private/auth {  
    mode = 0660  
    user = postfix  
    group = postfix  
  }  
}
```

10-auth.conf

In this file we must allow plaintext login. This is safe since we are running over a secure SSL encrypted link.

Find the following settings and adjust them as shown

```
disable_plaintext = no  
auth_mechanisms = plain
```

The above settings will allow a plain text username/password type of login to dovecot.



Note

Some **HowTo**s seem to indicate the use of LOGIN as the authentication mechanism for SASL. However, LOGIN is a deprecated mechanism and PLAIN is the recommended way as described in RFC 2595.

For the actual check of username/password Dovecot will make use of the PAM system on Linux which in turn will check any username/password against the Linux standard system users and password files (e.g. `/etc/passwd` or the shadow version of the password file)

10-mail.conf

The primary purpose of this file is to tell dovecot where in the file system mails are stored. The choice here is system dependent and up to the administrator. In the examples shown below mails are stored in `/srv/mail/%U` where `%U` is replaced by dovecot for the name of the user.

As for mail format we strongly recommend to use the maildir format where each mail is stored in its own file. To instruct dovecot to use this format we specify the mail location with a prefix of `maildir` as shown next. Now find and adjust the following setting

```
mail_location = maildir:/srv/mail/%u
```

We must also create this directory. Assuming we have a user called `tstuser` belonging to the user group `users` we can now create this mail directory with the proper ownership with the following commands:

```
$> sudo mkdir -p /srv/mail/tstuser
$> sudo chown tstuser:users /srv/mail/tstuser
$> sudo chmod 700 /srv/mail/tstuser
```

The above lines will create a mail directory only accessible by root and the specified user. The above steps are needed once for each mail user. In the above we have, somewhat arbitrary, chosen to store each users mail under the `/srv/mail` directory.



Tip

Another common setup is to store all the mails inside each users home directory, for example in `~/Maildir`

10-ssl.conf

Here we need to tell dovecot to enable the basic SSL capability. To do this the first line simply needs to change to

```
ssl = yes
```

10-logging.conf

The final step is to enable a more verbose logging. Strictly speaking this is not necessary but during the setup this is extremely handy in order to pinpoint any potential problems until we have a system that is fully working. Find the following settings and adjust them as shown.

```
log_path = /var/log/dovecot.err
info_log_path = /var/log/dovecot.info
debug_log_path = /var/log/dovecot.debug
auth_verbose = yes
auth_verbose_passwords = plain
auth_debug_passwords = yes
mail_debug = yes
verbose_ssl = yes
```



Tip

There is yet another level of logging that can be enabled to give details on every single IMAP command received from a client. This is very verbose and should only be enabled during specific troubleshooting sessions. Using this level of debugging requires some basic knowledge of the IMAP protocol in order to make sense of the information. This level of debugging is very useful if one suspects that a mail client sends the wrong IMAP commands (which is not completely unheard of)

This is described in the “Rawlog” section in the documentation of Dovecot (See <http://wiki2.dovecot.org/Debugging/Rawlog>)

Later on when things are working correctly these settings should be changed since it will generate quite a lot of “noise” in the log files.

5.2 Creating dovecot SSL/TLS certificates

In order to use IMAPS we must first create SSL certificates and keys in much the same way as we did for the Apache. In fact, it would be perfectly possible to reuse the exact same keys and certificates that we created for the web-server. Just watch out for the names of the certificate files.

However as we discussed previously we will use a separate set of certificates for Dovecot and postfix. For the purpose of creating certificates Dovecot includes a small shell script that will call all the necessary openssl commands in much the same way as **gensslcert** did for the web-server. This utility script is called **mkcert.sh** and is found in the directory `/usr/share/doc/packages/dovecot`. To get correct default values in the certificate first open and edit the file `dovecot - openssl.cnf` (that is in the same directory). This configuration file contains the values used for names and organization in the certificate. Open the file in an editor and change the setting to match the wanted setup.

Before **mkcert.sh** is run it is also a good idea to edit a line at the end of that file so that the generated certificate is valid for longer than a year which (unfortunately) is hard code in the end of the script. Change the argument “-days 365” in the end of the script to something a bit longer, for example 3650 days $\approx$ 10 years.

The files generated by this script are shown in Table 6, “Certificate keys generated for Dovecot”

Table 6: Certificate keys generated for Dovecot

Certificate file	Description
<code>/etc/ssl/certs/dovecot.pem</code>	SSL/TLS certificate. This certificate contains the public key for Dovecot.
<code>/etc/ssl/private/dovecot.pem</code>	SSL/TLS private key for the server. Must be kept private at all times!

As you can see this script will not generate any CA (root certificate). In fact the (test) certificate generated by the Dovecot utility script is by definition a root certificate in itself since it is signed by itself. This differs in behavior from the previously used **gensslcert** script when we configured Apache which created a separate root certificate. The configuration files in dovecot that specifies already have full path names as default so there is no need to change anything else.

Now it is time to restart Dovecot, for example by giving the command

```
$> rcdovecot restart
```

If you get an error saying something like

```
Fatal: execv(/usr/bin/doveconf) failed: Permission denied
```

Then our friend Novell AppArmour (which is included by default in a graphical OpenSuSE setup) has kicked in and stopped the process from starting. Since it brings too far to also configure AppArmour in this tutorial simply turn AppArmour off for now by opening Yast and go into Novell AppArmour control panel and disable it. For a personal server AppArmour usually creates more problem than it solves. By default it will block many useful services unless those services have been configured to be allowed.

5.3 Testing the dovecot configuration

To verify that the basics of our IMAP server works we can try to manually connect on port 993, however since that requires a setup of a SSL encrypted connection we cannot directly connect to the port (we need to use openssl s_client). So as an alternative we can (as a test) enable normal IMAP in `10-master.conf`. Do this by uncommenting the line `#port = 143` in the IMAP section at the top of the file. Now restart Dovecot in the same way as before.

We can then login to dovecot using a basic telnet connection

```
$> telnet localhost 143
Trying ::1...
Connected to localhost.Escape character is '^'.*
* OK [CAPABILITY IMAP4rev1 LITERAL+ SASL-IR LOGIN-REFERRALS \
ID ENABLE IDLE STARTTLS AUTH=PLAIN]
Dovecot ready.
```

The last line indicates that Dovecot is running and has accepted our connection. In order to do anything else we must login using a user in the system.

The IMAP login command is `login <user> <password>` So if you continue the previous telnet session by logging in using a known user and password (this user must exist in your system and should have a proper mailbox under `/srv/mail/`) you should now get a reply resembling

```
10 login tstuser tstpassword
10 OK [CAPABILITY ....] Logged in
```

If you wonder about the "10" in the beginning of the command this is just an arbitrary tag for the command. All IMAP commands has to be preceded by a user defined tag. When the IMAP server gives back a reply it will always use the tag for the command that caused this output. Tags allows us to know from which command an output comes from. It is possible to give several commands in a row and some might take long time so there is no guarantee that the servers reply will be in the same order as the given commands. Dovecot ignores the actual characters in the tag so it can be completely arbitrary.

The next step is to try list all available folders for this user

```
10 list "" *
* LIST (\HasNoChildren) "." INBOX"
10 OK List completed.
```

The above command also makes sure that dovecot can access the folder (in our case `/srv/mail/tstuser`)

If you get this far chances are high that the dovecot setup is correct. After we have configured postfix and we have some mails in our inbox we will manually retrieve them as well.

Finally disconnect from dovecot by logging out

```
10 logout
BYE Logging out.
10 OK Logout completed.
```

The final step remaining is to have Dovecot start automatically each time the server is started. If you haven't already done so give the command (as root):

```
$> chkconfig --add dovecot2
```

5.4 Verifying the encrypted IMAP connection

For this we need to use the command line tools from the openssl project which should already be installed. To connect to the server using SSL enter the command:

```
$> openssl s_client -connect localhost:993
```

This behaves in principle as a telnet connection with the only difference that the connection is encrypted. After the command is executed information is printed to the screen about the encryption and the certificates and at the end the words "Dovecot ready." should be visible. Now the same login test as for the non-encrypted connection can be run and then the list of available folders for the `tstuser`.

5.5 Troubleshooting Dovecot

More hints on how to debug a non-working Dovecot authentication process is described in the Dovecot troubleshooting documentation at:

<http://wiki2.dovecot.org/Debugging/Authentication>

If there is problem with mail synchronization after a successful login to Dovecot server there is one other level of logging (apart from the normal logging that can be setup) that can be helpful and that is to have the actual raw IMAP commands to be shown. This can be used when bug reports needs to be filed to non-compliant clients. To enable this one must activate the so called **rawlog** logging in Dovecot. How this can be enabled is described in detail in the Dovecot documentation at

<http://wiki2.dovecot.org/Debugging/Rawlog>

5.6 Adding a full text index

In order to drastically (~x1000) speed up full text searches (fts) of mailboxes it is possible to have Dovecot maintain a full text index for each mail folder. The easiest way to do this is to use Dovecots own full text search engine “SQUAT” (“Search Query Answer Tool”).

To add this capability we tell Dovecot to load this plugin by adding the line

```
mail_plugins = $mail_plugins fts fts_squat
```

in the file “20-imap.conf”. We must also globally enable these plugins and give some parameters in the file “90-plugin.conf”

```
plugin {  
  fts = squat  
  fts_squat = partial=4 full=8  
}
```

The index is initially created the very first time a search is performed on a directory so the first time will take extra time and memory depending on the number of mails in the directory. After the first time Dovecot will update the index incrementally as changes are made to the mail directories.

The only drawback enabling indexes is that they will use extra disk space. The needed space depends on the specific settings (see the Dovecot manual). But a minimum index size of ~30% of the total mail size in each directory can be expected. With the parameters as above one can expect ~60% index size. The parameters adjusts the tradeoff between speed of search and space usage.

6 Setting up postfix

To remind ourselves where we are in the overall setup process the setup of postfix will make it possible to send mails from our own machine by connecting to a running postfix daemon via the SMTP(S) protocol.

All relevant postfix configuration files can be found in the `/etc/postfix/` directory. In the first instance we will setup postfix using a regular SMTP connection without the added complexity of a SSL/TLS certificate. Once we have this working we will modify the configuration to use the same key and certificate we created for Dovecot. This will work just fine as long as Dovecot and the SMTP server are running on the same host. If they are running on separate hosts a separate and unique certificate for postfix must be created since each server with unique IP address must have its own certificate (to avoid warnings from clients connecting to them).

In postfix there are two files in the `/etc/postfix` directory. we need to update:

- `main.cf` - This is where all settings are stored, one huge parameter file
- `master.cf` - This file controls what helper processes are started and on what ports they are listening.



Caution

The changes you make manually in the postfix configuration might come into conflict if you later on decide to use Yast setup for the mail server. **So don't use the YAST setup once you have manually configured postfix** (into perfection!). Actually we would recommend against using Yast mail server setup for any serious mail server since the tuning possibilities are near to zero.

Also, please be careful when editing the configuration file and watch out for setting the correct server variable. Sender or receiver, i.e. with prefix `smtp_<variable>` or `smtpd_<variable>` this is a common source of errors that can be surprisingly time consuming to find.

6.1 User authentication in postfix

Before we continue we need to discuss how postfix handles user authentication. When do we need that? By default postfix restricts who can use our server to send mail based on IP address. It will allow clients on the same sub-network as the server. However, with mobile usage we will most likely connect to our postfix server through a broadband router and then the router will forward our request to the server. Hence the request looks like it is coming from our sub-network (our gateway) and will be allowed. This would then allow anyone that knows our official IP address to the gateway (and the port to use) to use our SMTP server to send mail that looks like it is coming from us! This is what is known as an open relay and is something really bad.

So, we need a way to authenticate users before they are allowed to send mail via the SMTP server. By itself postfix has no built-in method for this. Instead it is constructed to use an external library which implements an API known as SASL (Simple Authentications and Security Layer). The SASL library is a completely independent of postfix. There are several available providers of a compliant SASL library (for example the SASL authentication daemon `saslauthd`). The good news here is that Dovecot provides one possible implementation of a SASL library. Since we already have Dovecot installed it seems a good idea to use it. As you may recall we made some provisions for this when we configured Dovecot (even though we didn't explain the details at the time).

So after we have configured postfix to use the Dovecot SASL the following happens when a user connects.

1. Postfix send the authentication details through the SASL API to Dovecot
2. Dovecot receives the authentication details and uses the configured way to lookup the user details. By default dovecot uses the Linux PAM framework which normally is configured to check the user and password from the shadow password file. This requires all mail users to also have an account on the server. Dovecot can also use other methods such as an LDAP server or even a database with all the user credentials stored. The main advantage with storing users in a DB is that they do not need to have user accounts on the server.

3. Dovecot returns fail or success to postfix which based on this knows whether to accept or reject the incoming call.

There is still one problem with this authentication setup in that the user and passwords are sent in plain text over the network. For automatic network sniffers this is a gold mine since harvesting valid user/passwords combos is what spammers are after. The way to protect our conversation is to use SSL/TLS encryption of the communication between the client and the server in the same way we protected the IMAP and `https://` traffic.



Note

SSL (Secure Socket Layer) was renamed TLS (Transport Layer Security) by the IETF (Internet Engineering Task Force) as of version 3.1. There are some technical differences between SSL and TLS but for all practical reasons (as far as we are concerned) they are the same.

Configuring Dovecot SASL in postfix

Assuming we keep the network settings as default (allowing clients from the same sub-network to connect). We need to add/change the following settings in the postfix configuration file.

```
smtpd_sasl_auth_enable = yes
broken_sasl_auth_clients = yes
smtpd_sasl_type = dovecot
smtpd_sasl_path = private/auth
smtpd_sasl_security_options = noanonymous
```

Then add `permit_sasl_authenticated` to the `smtpd_recipient_restrictions` section as the following example shows

```
smtpd_recipient_restrictions = permit_mynetworks, permit_sasl_authenticated, \
reject_unauth_destination
```

The **order of these restrictions are important** as it is to finish with `reject_unauth_destination` since by default the connection will just “fall through” and be allowed !

Configuring postfix to use our ISPs SMTP server

If your server, like most home servers, does not have a fixed IP address you cannot send mails directly from your home server since all official mail servers on the internet must have a fixed IP address and have a so called, MX record in the DNS server which identifies the server as a **M**ail **eX**change server. Hence we must go through another proper mail server to send mail. This other mail server is usually our ISPs server (but it doesn't have to be). This is known as making our SMTP server a relay server (since it relays mail to this other proper mail server).

Most likely we also need to give a username/password in order for us to be able to login to our ISPs mail server. In the following we assume that our ISP:s mail server is called “`smtp.ourisp.com`” and accepts connection not on the standard port 25 but on port 26 (just to show how to handle other ports)

We must then add the following line in the configuration file

```
relayhost = [smtp.ourisp.com]:26
```

(The reason for putting the provider address in brackets is to avoid an MX lookup which can provide some problems. There are some valid (and good) reasons for this. But again, it would take too far outside this document to fully explain this.)

We also need to provide the correct user/password that our ISP has given us. In `main.cf` we point to a file where these credentials are stored.

```
smtp_sasl_password_maps = hash:/etc/postfix/sasl_passwd
```

and in the file `sasl_passwd` we add the line

```
[smtp.ourisp.com]:26 username:password
```

where the `username:password` are replaced by the proper values. Note that since both the `username` and `password` is in plain text this file should only be readable by root.

In order to update the corresponding `sasl_password.db` file run

```
$> postmap sasl_password
```

Finally we also need to restart (or reload) the postfix server

```
$> rcpostfix restart
```



Note

Postfix hash files.

As you can see above the filename is prepended with a “hash:” prefix. This tells postfix not to use the file directly but a database version of the file. This is done for efficiency reason and if you look in the `main.cf` file you will indeed see that all text files are accessed as hash files. This has one complication. If you update the source text file the database file also have to be updated. To update the `sasl_passwd.db` database file from the source `sasl_passwd` the utility toll 'postmap' is used:

Change into `/etc/postfix` directory and run

```
$> postmap sasl_passwd
```

This will create the `sasl_passwd.db` file that is used by postfix.

Verifying postfix SASL connection

Before moving on we should make sure that this works as we have intended. We can do this in the same way as we tested the IMAP connection, by hand. This allows us to see the core protocol level without any client getting in our way.

Unfortunately the SMTP protocol doesn't send the user/password combo as plain text as we did in the IMAP protocol testing. Instead SMTP expects the user/password to be base64 encoded together in one string. This can be manually constructed with the use of a one line perl script as shown below

```
$> perl -MMIME::Base64 -e 'print encode_base64("\000user\000userpasswd");'  
AHVzZXIAdXNlcnBhc3N3ZA==
```



Note

Strictly speaking (following the protocol) the username should be repeated so the really correct string would be

```
encode_base64("user\000user\000userpasswd")
```

but the simplified version will work. Another variant to do the base64 encoding is to use the Unix command '`mmencode`' which does Base64 encoding. This could be used as in

```
$> printf '\000user\000userpasswd' | mmencode
```

This is actually simpler than the Perl version but might not be available on all systems. **'mmencode'** can also be used to decode base64 encoded text.

where you replace the `<user>` and `<userpasswd>` placeholders as needed. It is then possible to use the result above when using telnet to login to the SMTP server and giving the `AUTH PLAIN` command.

```
$> telnet localhost 25
telnet localhost 25
Trying ::1...
Connected to localhost.
Escape character is '^]'.
220 myserver.com ESMTP Postfix
EHLO myserver.com
250-myserver.com
250-PIPELINING
250-SIZE
250-VRFY
250-ETRN
250-AUTH PLAIN
250-AUTH=PLAIN
250-ENHANCEDSTATUSCODES
250-8BITMIME
250 DSN
AUTH PLAIN AHVzZXIAdXNlcnBhc3N3ZA==
235 2.7.0 Authentication successful
quit
221 2.0.0 Bye
Connection closed by foreign host.
```

To make things clear the lines in bold face are the lines we entered. The rest of the lines are the replies from the SMTP server.

Make sure that you can see the line `250-AUTH PLAIN` since this is the line that indicates that the server accept authentication.

making an alias for root

We also want to have any system mail for root delivered to a “real” user. For this we need to setup an alias for root. This is done by making sure that the file `/etc/aliases` has a line

```
root: <user>
```

where `<user>` is a real existing user that will receive the root mails. After you have updated this file you need to run the command “**newaliases**” which will produce the file `aliases.db` based on the original alias file.

Rewriting sender addresses

If a mail is sent from the local user on a server that is not directly connected to the internet it will have a domain name that in most cases are “fake”, for example `<joe@localhost.nil>` (this will of course depend on what name you have given the server). The problem with this is twofold. First the recipient cannot answer to this address and secondly most mail servers requires that the from-address is a valid (can be looked up in DNS) to forward the mail at all.

To correct this situation postfix has the ability to rewrite your “bad” addresses to something the recipient can actually answer to. In fact, a lot of the complexity in postfix comes from the various ways of rewriting addresses. Here we will just use the most basic rewrite functionality. For a fuller (and very readable) discussion on the rewrite options in postfix consult the postfix HowTo “Postfix Address Rewriting [http://www.postfix.org/ADDRESS_REWRITING_README.html]” available on the postfix website.

This type of address rewriting is also known as address “masquerading” (for obvious reasons).

In order to have postfix translate the “bad” mail address to a “good” address a file with mappings between “bad” and “good” must be created. In this case we will use a map/file called “sender_canonical”. For example, assuming that “joe” want all his sent mail to look like it is coming from his GMail account <joe@gmail.com> we then add the following line to the sender_canonical file

```
joe      joe@gmail.com
```

After this we need to run the **postmap** (to create the DB file) and tell postfix to reload the configuration to make use of this new masquerading as

```
$> postmap sender_canonical
$> postfix reload
```

6.2 Setting up SSL/TLS for postfix

Installing certificates

As mentioned above we can use the same certificates we created for dovecot as long as postfix is running on the same server as Dovecot (same IP address).

The certificate and key we created for Dovecot were shown in Table 6, “Certificate keys generated for Dovecot”. We must now tell postfix to enable SSL/TLS for incoming connections as well as point postfix to those certificates that we want to use.

To do this we update `main.cf` as shown below

```
smtpd_tls_security_level = may
smtpd_tls_key_file = /etc/ssl/private/dovecot.pem
smtpd_tls_cert_file = /etc/ssl/certs/dovecot.pem
smtpd_tls_loglevel = 1
tls_random_source = dev:/dev/urandom
smtpd_tls_auth_only = yes
```



Note

The above line

```
smtpd_tls_security_level = may
```

replaces the older

```
smtpd_use_tls = yes
```

and tells postfix to use TLS.

For debugging purposes the loglevel can be increased to ‘2’.

The final setting, `smtpd_tls_auth_only = yes`, forces the use of TLS for SASL authentication and will not allow plain text authentication to occur unless a SSL/TLS session has been established. (It may therefore be useful to comment out `smtpd_tls_auth_only = yes` during testing.)

Accessing an encrypted server

Historically a separate port have been used for the encrypted version of the SMTP protocol when SSL was used. With SSL (and not TLS!) this is needed since the entire conversation is encrypted so the server listening on the

port has to use encryption from the start. The only real user noticeable difference between SSL and TLS is that with the modern TLS it is possible to start the conversation non-encrypted and as soon as both client and server has agreed to use encryption the rest of the conversation is encrypted. One of the big advantages with TLS is therefore that it is possible to use the same port for both encrypted and non-encrypted traffic (as defined by RFC 3207).

This means that we now have a choice on how we setup the actual access to the server over the encrypted connections.

- Use a dedicated port (old way) with SSL or
- Use the same port with TLS

For all modern installation it is strongly recommended to use the modern way with the same port using TLS.

Using a dedicated port for encrypted traffic

Again, this is **NOT** the recommended way but is included for completion. Unless you know that some clients only support this way you should not implement this.

Originally port 465 was used as the separate SSL port for encrypted SMTP or SMTPS (sometimes also referred to as SSMTP). Unfortunately this port assignment was never made official as far as IANA recognized port numbers go. Instead it is registered for URD – “URL Rendesvous Directory for SSM” by Cisco.

In addition the original intention of port 465 was to let SMTP servers forward mail between each other over an encrypted channel. It was never intended to be used by mail clients to talk to the server directly. For this purpose another port was to be used, port 587 “Message submission” (see RFC 2476).

So using this older setup an SMTP server was really supposed to be listening on three ports. Port 25 for normal unencrypted traffic both from other servers and from clients, port 465 to allow other servers to communicate and finally port 587 to allow mail clients to submit new mails.

However, reality doesn't necessarily always match the theory. Most servers only use two port, one for unencrypted and one for encrypted traffic. Even though technically incorrect most servers use port 465 for SMTPS (both from clients and other servers).

For a home based server (which does not have a DNS entry as a registered MX server connected directly to the internet as an official mail server) it doesn't really matter if you choose the common (but incorrect) port 465 or port 587 as long as you configure your clients correctly on which port they should use to send mails. The only thing you need to watch out for when using port 465 is that since this is not an officially assigned port for SMTPS one extra step is needed as described below.

Using port 465 for encrypted SMTP traffic

Since port 465 is not an officially recognized port for SMTPS we must either update `/etc/services` (that is a file which lists all officially recognized ports and gives symbolic names to them) to be able to use the symbolic name “smtps” for the port in our `master.cf` postfix or use the port number directly.

To use the symbolic name we must add the line

```
smtps    465/tcp
```

to `/etc/services` and remove any other service registered to that port. The entry in `/etc/services` is only made to be able to refer to the port number with a symbolic name.

The next step is to instruct postfix to listen on this port and enable the manager process that handles the TLS connections for postfix. This is done by editing `master.cf` and first removing the comment for the following line:

```
smtps    inet  n       -       n       -       -       SMTPD  -o smtpd_tls_wrappermode=yes
```

and then removing the comment for the line

```
tlsmgr    unix    -    -    n    1000?    1 \ tlsmgr
```

For these new settings to take place we now need to restart postfix

```
$> rcpostfix restart
```



Tip

As alternative you could skip the editing of `/etc/services` and just enter the numeric port number directly in `master.cf` but this is a more “hackery” way of administer a server. It’s best to have all the services recognized on the server listed in `/etc/services`.

Using port 587 for encrypted SMTP traffic

Since port 587 is an officially recognized port there is no need to update `/etc/services`. The only update needed is to remove the comments in `master.cf` for the two lines

```
tlsmgr    unix    -    -    n    1000?    1    tlsmgr
```

and

```
submission inet n    -    n    -    -    smtpd
```

This will enable the server to listen on port 587.

If you look in the `master.cf` file you will also see that by default some optional parameters are give directly on the command line. The parameters are mirrors of the ones given in the `main.cf` but will take precedence. The command line parameters are useful when you need different set of parameters for different ports.

Actually port 587 should really be used for submitting emails even if no encryption is used. Unfortunately many commercial providers do not follow this recognized standard and just assumes that both clients and other servers would contact them on port 25.

If you want to use port 587 for submission with only optional encryption for those client requesting this just remove the “`smtpd_tls_auth_only = yes`” option and SSL/TLS encryption will be optional.

Using the same port for both encrypted and non-encrypted traffic

This is the recommended way for a modern secure SMTP server setup.

With this method no extra ports are needed. Instead the normal port 25 is used for both encrypted and non encrypted traffic. When a client first connects to the server it will notice the by the servers response that TLS is supported (by the STARTTLS response code) and if the client is also capable of TLS encryption it might then ask the server to continue the rest of the conversation using SSL/TLS encryption.

To enable this method it suffices to enable the `tlsmgr` by uncommenting the line

```
tlsmgr    unix    -    -    n    1000?    1    tlsmgr
```

in `master.cf`

This might seem contradictory to what we recommended in the previous paragraph regarding the use of port 587. Technically you would be correct. The advantage is that some poorly written clients will only support port 25. In addition keeping as few ports open as possible is a good thing from a security point.

This is simply case where there simply is no single one correct answer. The official standard stipulates one thing but years of de-facto standard shows something else. Most (all?) ISP do not even allow incoming traffic on port 587 (which they really should!) and many clients only support port 25 for unencrypted submission and port 465 for encrypted submission (which technically is the wrong way of doing it).

Verifying the SSL/TLS setup when using a different SSL port

Finally we need to verify our setup. To verify that SSL/TLS is enabled in postfix we start by open a plain telnet session to postfix on the normal (non-secure) port 25

```
$> telnet localhost 25
telnet localhost 25
Trying ::1...
Connected to localhost.
Escape character is '^]'.
220 myserver.com ESMTP Postfix
EHLO myserver.com
250-myserver.com
250-PIPELINING
250-SIZE
250-VRFY
250-ETRN
250-STARTTLS
250-ENHANCEDSTATUSCODES
250-8BITMIME
250 DSN
quit
221 2.0.0 Bye
Connection closed by foreign host.
```

The thing to notice above is the absence of `AUTH PLAIN` (since we disabled that unless we had a secure connection which we don't have since we connected with telnet) and the addition of `250-STARTTLS` which indicates that postfix is ready to accept a secure connection over TLS

The rest of the section assumes that the server has been setup to listen on port 465 for secure connections.

The next step therefore is to verify the encrypted connection in the same way as we did for the IMAP server using the “`openssl s_client`” command (see `s_client(1)` for more details).

```
$> openssl s_client -connect localhost:465
```

This will first give you a long list of SSL/TLS details and in the end you will see a line like

```
220 smtp.myserver.com ESMTP Postfix
```

You can then give the `EHLO myserver.com` command to see what capabilities are supported and it should be the same as in the previous telnet session but without the `STARTTLS` line since we are already in a TLS session but with the `AUTH PLAIN` (and `AUTH=PLAIN`) indicating that postfix is ready to accept an incoming login. After this you should also try to login in the same way we did without SSL/TLS (just remember the base64 encoding).

Once this is working you should try to access the SMTP server using the full IP address or hostname, for example as

```
$> openssl s_client -connect 192.168.0.50:465
```

It is possible that this will fail. In that case it most certainly is caused by you having the following line

```
inet_interfaces = localhost
```

In `main.cf`. This line tells postfix to only accept incoming connection only to localhost and not the IP address. Recall that the network card can be accessed with (at least) two IP addresses first the loopback address

127.0.0.1 (or localhost) and then the assigned IP address (in our case 192.168.0.50). Either remove the line completely or add the IP address of the host to the line, i.e. in this case. As a general rule you should try to clean up your postfix configuration file to only contain settings you have made yourself and then comment them heavily, (the shorter the configuration file is the easier it is to understand).

To fix this you could remove the line or add the IP address of the server as

```
inet_interfaces = localhost, 192.168.50
```

(Leaving `inet_interfaces` empty or setting it to “all” will accept connection on all defined interfaces on this machine.) With the above use of `openssl s_client` you will get a certificate warning that openssl can not verify the root certificate. This is to be expected since we have not told openssl what root certificate we trust. In our case this warning is harmless but to avoid this warning add the parameter `-CAfile </path/to/rootcert/ca.crt>`

If you have followed the previous recommended Apache setup then one place to find our root certificate is

```
$> openssl s_client -CAfile /etc/apache2/ssl.crt/ca.crt -connect 192.168.0.50:465
```

Verifying the SSL/TLS setup when using one port for both encrypted and non-encrypted traffic

This also requires the use of the `openssl` with the `s_client` command.

However since the connection starts out non-encrypted and must be enabled by following the SMTP protocol we must add an extra option `-starttls smtp` to tell openssl to obey the SMTP protocol as

```
$> openssl s_client -starttls smtp -connect 192.168.0.50:25
```

The rest of the tests can be done in the same way as in the previous section.



Tip

For more complex testing of SMTP transaction we recommend the program “swaks” which stands for *Swiss Army Knife SMTP*. This is a scriptable tool which simplifies complex SMTP testing SMTP including SSL/TLS. See <http://www.jetmore.org/john/code/swaks/>

6.3 Enabling postfix at boot time

Finally to have postfix start automatically when the server is started we use the '`chkconfig`' command

```
$> chkconfig --add postfix
```

7 Setting up fetchmail

Fetchmail's role is to fetch mail from remote hosts to our server and deliver the fetched mail locally. In order to fetch mail we must first setup a configuration file for fetchmail with the remote servers and there credentials where we want to fetch mail, this configuration file is `/etc/fetchmailrc`.

After fetchmail has logged in it will deliver the mails locally to our SMTP server just as if the mail had come from some remote SMTP server connecting to us.

There is one thing to keep in mind here though and that is that fetchmail only handles password when fetching mail. It cannot handle passwords in the local delivery. This means that our postfix setup must accept incoming mails that should be delivered locally without password (which is the normal). Since we have specified `smtpd_recipient_restrictions` with the option `permit_mynetworks` This will allow fetchmail to store mail locally without providing a password.

To be really secure we should restrict this further by only allowing access from within the server itself. This can be done by making sure that `mynetworks_style=host` is set and make sure that `mynetworks` is empty since that configuration variable has precedence (if that is set the `mynetworks_style` will be ignored).

7.1 Configuring fetchmail

In the documentation for **fetchmail** there are several examples on how to setup a basic configuration. So in this section we will show some examples of more complex setup which isn't available in the fetchmail documentation.

The advantage (if any) with fetchmail configuration syntax is that it is quite "chatty" and can be read almost as an English sentence.

Let's start with a very basic example. Assuming that the remote server is called "remotemail.com" and the server uses POP3 (not IMAP). Furthermore we assume the username is "joe" (both remote and local) and the password is "secret".

The following configuration line will handle this case:

```
poll remotemail.com proto pop3 user "joe" there \  
with password "secret" is "joe" here;
```

Let's now do a slightly more complex setup. In this setup we are connecting to an IMAP server over SSL/TSL (meaning that we need to connect on port 993) and we want to collect mail from both the "Inbox" folder as well as the "Junk" mail folder back to our server.

Furthermore we want to permanently delete mails that are rejected as spams by our own postfix server when fetchmail tries to deliver it.

```
defaults  
    user "joe@remotemail.com" is "joe"  
    set no softbounce  
    antispam 501  
    bad-header accept  
  
poll imap.remotemail.com with proto IMAP port 993:  
    SSL  
    folder 'inbox','Junk'  
    password "secret";
```

There are a couple of unusual thing in this setup.

- To make it more readable we first set up some default values that apply to all defined servers. We use this to first set the user we are collecting mail from. The second option requires some more explanations. If we fetch a mail from the remote server and when we try to deliver it to our own postfix server it is refused since it is

considered a wrongly formatted mail. In this case fetchmail will normally leave the mail on the remote server. With this option set fetchmail will remove the mail from the remote server even when it was rejected by our own server, effectively deleting the mail without anyone ever receiving it.

This might sound like a bad thing to do. However mail that are so wrongly formatted that postfix refuses to accept it as a proper mail are to 99.999% spams.

- The `antispam 501` line tells fetchmail to interpret an 501 error (Bad address syntax) from the local postfix server not as an error but as a signal that the delivered mail is interpreted as a spam by postfix. It will then discard the message and since “`set no softbounce`” is in effect. The mail will then also be deleted from the remote server.

A common cause of this is that postfix with default settings refuses addresses which begin by an “-“ (hyphen). This might be unusual but it is actually allowed (see RFC 5321, RFC 5322) so in this case postfix is a bit picky.

- The “`bad-header accept`” command allows **fetchmail** to collect mail even if some of the headers are incorrect.
- We then specify that we connect to the IMAP server on port 993 and enable SSL.
- To get mail from several folders we use the “`folder`” command and then just list the folders we want to collect the mail from

Before starting fetchmail check that you are happy with the default configuration in `/etc/sysconfig/fetchmail` such as poll interval. (Just keep in mind that some ISP gets a bit unhappy if you try to poll there server every minute or so.)



Tip

There is also a GUI tool **fetchmailconf** that can be used to ease the creation of fetchmail configuration files. You can install this with

```
$> sudo zipper install fetchmailconf
```

7.2 Starting and enabling fetchmail at boot time

To manually start **fetchmail** give the command

```
$> chkconfig --add fetchmail
```

You should also add fetchmail as a service that is started at the same time the server boots. You can do this via Yast or by giving the command

```
$> chkconfig --add fetchmail
```

This command will start fetchmail in the default run levels (most likely 3- multiuser with network and 5 - full X-window client)

8 Setting up spamassassin

Spamassassin is a Perl based program/script that uses statistical analysis of text to determine the likelihood that a certain mail is a spam or legitimate mail. Spamassassin is one of the oldest spam filter but has been regularly updated and is good enough for small scale home based server. The main complaints for a large scale setup of is not the quality of the analysis but rather that it requires quite a lot of computing power and can take several ms to scan each mail. In a large (several thousand mails an hour) setup this can become a bottleneck and a separate server might need to be deployed for this purpose..

However there is one way to speed things up and reduce the system load and that is the **spamd/spamc** daemon/client setup. This allows spamassassin to remain running in the memory and avoids starting up a Perl process and shut it down for each mail to be scanned. This is also installed by default (but not enabled). To call the resident daemon the client **spamc** is used. This is also the client program we call from within our procmail script to run the mails through the spam filter.

To enable the **spamd** daemon either enable it via Yast System Services (runlevel) editor or via the command line

```
$> chkconfig --add spamd
```

After installing spamassassin via Yast there are several places where the configuration files reside.

- `/usr/share/spamassassin` - This is where all the static rules are stored that gets installed with spamassassin. In here is also the file `local.cf` which is the only file that should be modified for local preferences since all other files will be overwritten in case of an upgrade. However it is recommended that you do not make any modifications here. Instead use the config file under `/etc` (see next bullet)
- `/etc/mail/spamassassin/local.conf` - This is only the proper place to do local modifications for server wide adjustments
- `<user home>/.spamassassin/user_prefs` - This is the proper file to set user specific modifications

When spamassassin reads the configuration files it first reads the files under `/usr` then the files under `/etc` and finally the users home directory. If any file has the same name the later version will be added to the previous version so any changes in a later file will override previous changes.

The default settings are usually good enough for the majority of users but you may want to lower the default spam threshold of a score of 5.0 to something a bit lower, say 4. This change could be made in `/etc/mail/spamassassin/local.conf` to affect all users on the server.

It is important to understand that spamassassin by itself does not sort mails in any way. It simply adds a few headers in the mail indicating how likely the mail is to be a spam. It is then up to the rules in the procmail script to interpret these headers to determine if the mail should be put in the spam folder or be received as a regular mail. The specific headers added by spamassassin are shown in table Table 7, “Headers added by spamassassin”.

Spamassassin is in our setup run by procmail as an external program and run as the user who is receiving the mail. Procmail puts each mail whose size is less than a predetermined size through spamassassin and then looks in the headers to determine if the mail should be put in the spam folder, possibly deleted forever or just forwarded to the normal inbox or perhaps some other mail folder. Review the procmail configuration in the next section for how this is done.

The reason to limit the size of mail we put through spamassassin to avoid having (for example) large mails with multimedia content to be run through spamassassin since that would consume a lot of time and spam mails are never (well almost) that large so that would just be a waste of server time.

8.1 Configuring spamassassin Bayes auto learning

Apart from the static analysis on headers and content of the mail spamassassin also has the ability to learn the particular words in typical spams. This later on helps spamassassin identify spams.

Bayes auto learning is enabled by default but you should also check that it is by checking that `bayes_auto_learn 1`

By default spamassassin learns word from mails and learns them as either spam or non-spam ("ham") depending on thresholds defined in the configuration file. By default these thresholds are rather high to avoid learning from false positive. You can be more aggressive here and for example use the following lower thresholds values.

```
# Use Bayesian classifier (default: 1)
#
# use_bayes 1

# Bayesian classifier auto-learning (default: 1)
#
# bayes_auto_learn 1

# Mail which scores outside this range will be fed back into SpamAssassin's
# learning system automatically, to train the Bayesian scanner.
ifplugin Mail::SpamAssassin::Plugin::AutoLearnThreshold
bayes_auto_learn_threshold_nonspam    0.01
bayes_auto_learn_threshold_spam      6.0
endif # Mail::SpamAssassin::Plugin::AutoLearnThreshold
```

The above changes can be made either in `/etc/mail/spamassassin/local.conf` to take affect side wide or in the `~/ .spamassassin/user_prefs` file to affect only a specific user.

If you have a folder with eihter spam or ham you can also instruct spamassassin to learn from that collection with the use of the "**sa-learn**" command. For example as

```
$> sa-learn --spam <folder/with/spam.mails>
```

8.2 Verifying spamassassin

To verify that the spamassassin daemon is running we can do a simple manual test by running some nonsense text through the spamassassin client **spamc** which in turns calls on **spamd**.

```
$> echo "Nonsense text" | spamc -R
```

You should now get a fairly long reply with the headers in table Table 7, "Headers added by spamassassin" clearly visible in a format that (almost) looks like the raw format of a mail.

Table 7: Headers added by spamassassin

Header	Description
X-Spam-Checker-Version:	Version information Example: <code>X-Spam-Checker-Version: SpamAssassin 3.3.1. (2010-03-16) on myhost.nil</code>
X-Spam-Flag:	A flag YES or NO to indicate if the computer score is over the set spam threshold Example: <code>X-Spam-Flag: YES</code>
X-Spam-Level:	A graphical indication with ‘*’ to indicate the computed spam score. Each ‘*’ represents one integer score. Example: <code>X-Spam-Level: *****</code>
X-Spam-Status:	A detailed list with the numerical score, required score and list of tests that indicates that this might be a spam. Example: <code>X-Spam-Status: Yes, score=8.0, required=4.0 tests=EMPTY_MESSAGE, MISSING_DATE,MISSING_MID, NO_RELAYS autolearn=no version=3.3.1</code>

9 Setting up procmail

When postfix receives a mail it will by default store the mail in the mail spool directory (either “/var/spool/mail/user” or “/var/mail/user”).

Postfix offers no possibility of post-processing of the received mails. For example to run the received mails through a spamfilter (like spamassassin) and put all identified spams in a separate mail folder.

To solve this we tell postfix to send all the received mails that should be locally delivered through another program, in our case procmail. It is this program that will act as a Local Delivery Agent, LDA, and perform the last stage when incoming mails are store on disk.

9.1 Configuration file

Procmail is controlled with (yet another) configuration file which by default is in the home directory of the user receiving the mail (.procmailrc). The configuration file will contain rules on how to process the incoming mails, such as storing mails based on different criteria in different folders. It is even possible to have certain mails automatically trigger other external programs. A typical use of this would be a server where users can send a one line mail to have some useful information automatically sent back.

In order to tell postfix to send incoming mails to procmail we must add the following line in the postfix config file (/etc/postfix/main.cf)

```
mailbox_command = /usr/bin/procmail
```

The next step is to setup the configuration file for procmail. Since it is possible to write an entire book on how to write procmail filtering rule we will keep this example simple. The example rule file below will do the following:

- we will enable logging in a separate log file for each user under the /tmp directory
- if the mail is judged to be “normal” spam we store it in the folder “Junk”
- if the mail have a very high spam score we just discard them by deleting them forever
- if the mail has the word “Newsletter” in the subject or comes from a sender “*.news.com” we put it in a separate mail folder “Newsletters”
- otherwise we store it in the default inbox

The home page for procmail has many more receipts that you can take directly. For example “*Out of office*” style of automatic replies.

Unfortunately the syntax for procmail configuration is not exactly intuitive (anyone who think that has spent too much time with mail servers) but the basic structure in this example file will get you a good start and will serve a good start for your own experiments. It would bring this tutorial too far off track to become a “How To Write procmail Filter rules” as well. Refer to the procmail documentation for details.

In Appendix B, *Procmail recipes* we also show some more example of procmail configuration recipes.

Example 1: Template for a procmail configuration file

```
#-----
# Standard setup for extra logging
#-----
VERBOSE=on
LOGABSTRACT=all
LOGFILE=/tmp/procmail-${LOGNAME}-`date +%Y%m`.log
MAILEDIR=/srv/mail/${LOGNAME}
DEFAULT=${MAILEDIR}/
INBOX=${MAILEDIR}/
```



```

#-----
# Subfolders to filter mails in.
# REMEMBER TO PUT TRAILING '/' AT # THE END
# to mark the folders as being in maildir format
#-----
SPAM=${MAILDIR}/.Junk/
NEWSLETTERS=${MAILDIR}/.newsletters/

##-----
## End of setup. Now comes all the filter
## rules.
##-----

#-----
# FILTER for Newsletters.
#-----
:0
^Subject:. *Newsletter
^From:. *news.com
{ }
:0 E
$NEWSLETTERS

#-----
# FILTER for spamassassin
#-----
:0
* < 512000
| /usr/bin/spamc
#-----
# Discard all spams with score >= 15
#-----
:0
* !^X-Spam-Level: \*\*\*\*\*\*\*\*\*\*\*\*\*\*\*\*.*
* !^X-Spam-Status: Yes, score=1[5-9]\.[0-9]
* !^X-Spam-Status: Yes, score=[2-9][0-9]\.[0-9]
{ }
:0 E
/dev/null

#-----
# Put all remaining spams in Junk folder.
# NOTE: This assumes spamassassin is configured
# to set the X-Spam-Flag: YES for identified
# spams.
#-----
:0
* ^X-Spam-Flag: YES
$SPAM

#-----
# Send the remaining mails to the inbox
#-----
:0
$DEFAULT

```



Note

By default procmail only supports “AND” rules, i.e. if you add a number of conditions before an action all of them has to be fulfilled for the action to be performed.

One of the most frequent questions newcomers to procmail have is therefore how to write “OR” rules, i.e. a rule that states that

if condition1 or condition2 is true then do action

The way around this is to realize that

`"if A or B then do C"`

is equivalent to

`"if (not A) and (not B) then do nothing else do C"`.

Any book on Boolean logic will give more details on this subject.

10 Setting up Roundcube

(Note: The following section assumes that roundcube v0.7 and MySQL community server have been installed, see Section 1.2, “Applicable OpenSuSE versions and pre-requisites”)

We have finally come to the part that users will see; the Web-based mail client. To setup Roundcube do the following steps:

1. Download Roundcube from <http://roundcube.net/>
2. Extract the sources to your SSL enabled http document root. We also recommend to change the main directory name for Roundcube to something more generic, say "webmail". Using the structure suggested previously in this document this would then be `/srv/www/ssl-htdocs/webmail` and will make it possible to access your mail in a natural way as `https://localhost/webmail/`



Tip

One alternative way of placing Roundcube sources is outside the document root, say `/usr/local/share`. Then we create an Apache alias to that directory. This means that if the user accesses (for example) `/webmail` Apache knows that it should actually fetch the content from the directory `/usr/local/share/roundcube` assuming we have setup our aliases in that way. Please refer to the Apache documentation for more details on the `alias` command. This is a good practice to avoid installing Web-applications directly in the document root.

3. Make sure that the Web-server can write to ‘temp’ and ‘logs’ directories within the extracted files in the document root by executing the following command as root

```
$> chown -R wwwrun:www temp logs
```

4. Setup the MySQL user and database that roundcube should use as follows (this assumes that you previously have installed MySQL and have set a password for the `root` user.)

```
$> mysql -h localhost -u root -p
CREATE DATABASE roundcubemail;
GRANT ALL PRIVILEGES ON roundcubemail.* TO roundcube@localhost \
IDENTIFIED BY 'roundcube';
FLUSH PRIVILEGES;
```

Note: You can change the user and password to something better but since the database is only accessible from localhost it is still quite secure.

5. Point your browser to the installer directory <https://webmail/installer/>

The installation is a three step process. The first step is a verification that the PHP5 installation has all necessary modules installed. Verify that all the necessary prerequisites are fulfilled.

In the next step the details of the database, the IMAP and SMTP settings must be given. Below are some examples on how this can be specified (change user and passwords according to your settings)

Table 8: Roundcube MySQL settings

Field	Value
Database server	localhost
Database name	roundcube
Database user name	roundcube
Database password	roundcube

Table 9: Roundcube IMAP settings

Field	Value
default_host	ssl://localhost
default_port	993

Note: Make sure that `auto_create_user` is marked since it will be impossible to login otherwise!

Table 10: Roundcube SMTP settings

Field	Value
smtp_server	tls://localhost
smtp_port	25

Note: Please leave the SMTP username and password empty. Roundcube will then use the login information to access the SMTP user.

In the next step the installer will show the two configuration files "`main.inc.php`" and "`db.inc.php`". Download them and store them in the `config` directory.

The final step in the setup is a test to make sure that the settings are correct. Make sure that **all** these tests succeeds before proceeding!

6. It should now be possible to point your browser to the directory where you have installed Roundcube (`https://localhost/webmail/`) and see the interface. After making sure that this works the installer directory should be removed.

11 Where to find more information

Seeing that this description has taken far too many pages and still hardly can be called complete one has to realize that it's not completely trivial to setup a fully working mail server. There are many sidetracks we have avoided or just scratched on the surface on to keep this HowTo from becoming a fully-fledged book. However we hope that this description has provided a good overview and points of interest worthy of further study. Below we have listed some sources of information for the different tools we have described here.

11.1 On the usage of SuSE Yast tool

Using Yast might at times seem tempting however the setups possible via Yast are limited and doing anything just outside the very basic setup is often not possible. In addition some necessary configurations comes from manipulating `/etc/sysconfig` file where documentation is scarce.

For that reason one has to manually configure most setups. In addition there are some differences in the SuSE setup files compared with some tools default setup. In most cases these changes are done to improve upon the standard delivered files. However lack of proper detailed documentation makes it necessary to very carefully read through a whole lot of system and internal configuration files that SuSE uses (but no-one else) to really understand, for example, how a program is started.

In addition there are several utility scripts available that can help with mundane tasks but since they are not documented (or that the documentation is successfully hidden!) it's difficult to know about them.

11.2 References

1. "Survival guides - SSL/TLS and X.509 (SSL) Certificates", <http://www.zytrax.com/tech/survival/ssl.html>
2. "Apache 2.1/2.1 manual", <http://httpd.apache.org/docs/2.1/>

Appendix A - Postfix configuration examples

One key to configure postfix SMTPD access is to understand the configurations directive `smtpd_recipient_restrictions`. Below are two examples on how this can be used. Remember that the order of the settings in this configuration is important. Postfix will execute them sequentially

A.1 Example 1

The default settings will only allow anyone on the same subnetwork as the server to send relay mail through

```
mynetworks_style = subnet # only allow clients on the same subnet to connect

smtpd_recipient_restrictions =
    permit_mynetworks
    reject_unauth_destination
```

A.2 Example 2

Changing the `mynetwork_style` to only allow password less logins originating from the same server and require (SASL) password from all other logins are safer and can be done by this

```
mynetworks_style = host

smtpd_recipient_restrictions =
    permit_mynetworks
    permit_sasl_authenticated
    reject_unauth_destination
```

A.3 Example 3

Allow clients on the local machine (host) to connect and send relay mails without logging in. Force external clients to use SSL/TLS before allowing them to login.

```
mynetworks_style = host

# sasl
smtpd_sasl_auth_enable = yes

# tls
#
smtpd_tls_security_level = may
smtpd_tls_loglevel = 1
smtpd_tls_cert_file = </path/to/crt.crt>
smtpd_tls_key_file = </path/to/key.key>

# Only expose the AUTH command after an encrypted SSL/TLS
# session is established
smtpd_tls_auth_only = yes
smtpd_tls_session_cache_database =
    btree:/usr/local/etc/postfix/smtpd_tls_scache

smtpd_recipient_restrictions =
```

```
permit_mynetworks
permit_sasl_authenticated
reject_unauth_destination
```

A.4 Example 4

This setup requires password authentication from both local and remote users but only require SSL/TLS from remote users

```
#being strict on mynetworks isn't important for this setup, but:
mynetworks_style = host

# sasl
smtpd_sasl_auth_enable = yes

# tls
#
smtpd_tls_security_level = may
smtpd_tls_loglevel = 1
smtpd_tls_cert_file = </path/to/crt.crt>
smtpd_tls_key_file = </path/to/key.key>
smtpd_tls_session_cache_database =
    btree:/var/lib/postfix/smtpd_tls_scache

smtpd_recipient_restrictions =
    #allow anyone delivering mail to this host:
    permit_auth_destination
    #otherwise, allow authenticated connections from localhost
    check_client_access hash:/usr/local/etc/postfix/access_client
    #otherwise, demand both authentication and encryption:
    reject_plaintext_session
    permit_sasl_authenticated
    reject
```

/usr/local/etc/postfix/access_client should look like this:

```
# connections from 127.x.x.x must authenticate
127 permit_sasl_authenticated, reject
```

Appendix B - Procmail recipes

The first line of a procmail filter always starts with “:0” and the remaining letters controls the filtering process.

For example “h” will only filter on the mail headers, “c” will make a copy of the mail so that it is available to filters after the current filter. If there is no “c” then the filter will consume the mail and processing stops here and no more filters will be applied to the mail.

Five good sites to find out more about how to write procmail recipes are:

- <http://pm-doc.sourceforge.net/doc/> (The official procmail documentation)
- <http://partmaps.org/era/procmail/mini-faq.html> (Procmail FAQ)
- <http://www.ii.com/internet/robots/procmail/qs/>
- <http://userpages.umbc.edu/~ian/procmail.html>
- <http://lipas.uwasa.fi/~ts/info/proctips.html>

B.1 The problem of filtering on the “Subject:” field

While at first it might seem simple enough to filter out certain mails with a specific subject either as spam or some other reason it is in practice a lot more complicated. The problem is that the subject does not always have to be in plain text. Instead it is very common to have the subject in either quoted printable or in base64. Such an encoded string looks like `=?charset?format?encoded text?` (this format is more specifically described in RFC1522) and it can also be followed by some text in plain format and then again by some encoded text. For obvious reason a lot of spam uses this format to make it much more difficult to do simple text matching.

Unfortunately there is no easy way around this and procmail does not have any built in “decoding” functionality so this has to be done manually.

To help with this the following small perl script can be used as a helper script. This script has been floated on the net and the author is unfortunately unknown to me (if you know please let me now and the proper credit will be given)

```
#!/usr/bin/perl
#
# Perl script to decode a subject string possible encoded with RFC1522
# quoted printable or base64 encoding.
# Author: Unknown.

while (<>) {
    chomp;
    s/^[ \t]//;
    foreach (split(/(=[?].*?[?][BQ][?].*?[?]=)/i)) {
        if (($charset, $encoding, $txt) = /(=[?](.?[?])([bq])[?](.?[?])?[?]=/i) {
            $encoding =~ tr/[BQ]/[bq]/;
            open PIPE, "echo '$txt' | mimencode -u -$encoding |";
            $_ = <PIPE>;
            close PIPE;
            chomp;
        }
        print $_;
    }
}
```

Store this script a file and make the file executable (in the example below we will assume it is stored in `/usr/bin/decode_subjheader`) In procmail we can then use formmail to first extract the subject header and then run that header through this perl script to get the decoded subject

For example the helper script can be used as:

```
SUBJECT = `formail -xSubject: | /usr/bin/decode_subjheader`
:0
* SUBJECT ?? (Newsletter.*)
$NEWSLETTER
```

The recipe above will first extract the subject header from the mail and run it through our decoder perl script and then store the decoded header in the variable SUBJECT.

The details of the filtering rules are explained in Table Table B.1, “Explanation of filtering rules”

Table B.1: Explanation of filtering rules

<code>:0</code>	Begin a new rule
<code>*</code>	Begin a new condition
<code>SUBJECT ??</code>	Match the variable SUBJECT to
<code>(Newsletter.*)</code>	... any subject line starting with “Newsletter”
<code>\$NEWSLETTER</code>	If the filter matches then store it in the folder pointed to by the content of the variable

B.2 How to create filter that matches on any of several rules (creating an “OR” rule)

By default all matching rules in a filter need to apply before the action is taken. To create a rule which simulates an “OR” condition for the matching expression one has to make use of the “else” rule. This is done by executing an empty action “{ }” when all the conditions do NOT match and our “real” actions if this condition is not true. (This uses the fact that $\text{NOT}(\text{NOT } A) \text{ AND } (\text{NOT } B) = A \text{ OR } B$)

```
:0 hb
* !<expression 1>
* !<expression 2>
{ }

:0 E
<action to be taken>
```

Please note that there should be one space in the “{ }” empty statement!

So for example to store mail that comes from either “<joe.r@gmail.com>” or “<helen.q@gmail.com>” in a folder called “Friends” the following recipe could be used

```
0: hb
* !^From:. *joe.r@gmail.com
* !^From:. *Helen.q@gmail.com
{ }

:0 e
$FRIENDS
```

B.3 Out of office

The following recipe shows how to send an automatic answer to incoming messages. For this we use the utility program “formmail”. We also take some precaution to avoid an infinite loop when the sender also has an automatic reply to avoid mails just bouncing back and forth.

```
#-----
# Out of office
#-----
:0 h c
* !^FROM_DAEMON
* !^FROM_MAILER
* !^X-Loop: out_of_office
| (/usr/bin/formmail -r -A"Precedence: junk (autoreply)" -A"X-Loop: out_of_office" \
-I"Subject: Out of office reply"; cat $HOME/out_of_office.txt) | $SENDMAIL -t
```

B.4 Replying with a mail based on subject

This shows how to use formmail to perform an action based on the subject of an incoming mail. In this particular case we respond to a specific mail by powering on a remote server

```
#-----
# Sending a reply based on subject
#-----
:0 h c
* ^Subject: gwake
| (/usr/bin/formmail -r -I"Subject: Server gamma waking up (autoreply)" ; \
echo "This is not a guarant\
```

```
ee that the server has woken up though. " ) | $SENDMAIL -t
```

B.5 Calling an arbitrary external program

This shows a simple variant of calling an external program based on the subject in a mail

```
#-----
# Start up gamma server
#-----
:0
* ^Subject: Send information
| /usr/bin/the program
```

B.6 A template to start with

The final example is not really an example but a template that you may use to get a good start in writing your own rules. It give a logical structure and establishes some "best practices" to get readable and understandable procmail files (as far as that is possible). This template will give you

- A logfile under /tmp for each month
- Spamfiltering based on spamassassin with both filtering to a "Spam" folder and also direct deletion of mails with a high spam score (>=15)
- A good practice of giving symbolic names to the mail folders where to store mail

```
#-----
# Standard setup for extra logging
#-----
VERBOSE=on
LOGABSTRACT=all
LOGFILE=/tmp/procmail-${LOGNAME}-`date +%Y%m`.log

#-----
# Some standard paths
#-----
MAILDIR=/srv/mail/${LOGNAME}
DEFAULT=${MAILDIR}/
INBOX=${MAILDIR}/

#-----
# Subfolders to store mails in. You should add
# a variable for each folder you are using in your
# mail setup. The wasiest way to create the proper
# folders is to do it via you email client,
# REMEMBER TO PUT TRAILING '/' AT THE END
# to mark that the folders are in maildir format.
#-----
SPAM=${MAILDIR}/.Junk/
NEWSLETTERS=${MAILDIR}/.newsletters/

##-----
## End of setup. Now comes all the filer
## rules.
##-----

#-----
# FILTER all Newsletters.
#-----
:0
```

```

^Subject:. *Newsletter
^From:. *news.com
{ }
:0 E
$NEWSLETTERS

#-----
# FILTER for spamassassin. Only run mails smaller than ½ MB
# through spamassassin
#-----
:0
* < 512000
| /usr/bin/spamc

#-----
# Delete all spams with score >= 15
#-----
:0
* !^X-Spam-Level: \*\*\*\*\*\*\*\*\*\*\*\*\*\*\*\*.*
* !^X-Spam-Status: Yes, score=1[5-9]\.[0-9]
* !^X-Spam-Status: Yes, score=[2-9][0-9]\.[0-9]
{ }
:0 E
/dev/null

#-----
# Put all remaining identified spams in Junk folder.
# NOTE: This assumes spamassassin is configured to
# set the X-Spam-Flag: YES for identified spams.
#-----
:0
* ^X-Spam-Flag: YES
$SPAM

#-----
# Store the reaming mails to the inbox
#-----
:0
$DEFAULT

```

Appendix C - Dovecot tips and tricks

C.1 Automatic expunging of old mails

Subscribing to non-personal mail like newsletters and mail lists tends to grow the mailbox quite a lot with information that usually is valid only for a short time. So having this kind of mail automatically deleted after a certain time period, say 2 weeks, is usually a good thing.

Dovecot provides a utility program called `doveadm` that can do exactly this. Among other things `doveadm` have a command `"expunge"` to delete all mails that matches a certain search criteria in a specified mailbox.

An example will clarify how this work. Assume we want to delete all mails older than two weeks in our "dovecot" mail folder. To do this we search and delete all mails that were stored in that mail folder more than fourteen days ago with the command:

```
/usr/bin/doveadm expunge mailbox dev.dovecot savedbefore 14d
```

All messages older than 14 days in the mail folder "dev/dovecot" have now been deleted.

In Linux we can then setup a cron job that calls a script with these expunge command at regular intervals, say once a week or even once a day.

The search commands for `doveadm` are quite flexible so please look at the man page for other search options. The above line will cause all mails (both read and unread) to be deleted. One common modification is to have non read messages kept longer.

If we store the above line in a script as

```
#!/bin/sh
/usr/bin/doveadm expunge mailbox dev.dovecot savedbefore 14d
```

and store the script as, say, `/usr/local/bin/user-expungemail` we can then use a cronjob to call this script at regular intervals. Having the jobs called every Wednesday and Sunday at 3 a'clock in the morning would for example require us to write

```
0 3 * * 3,7 /usr/local/bin/user-expungemail
```

If the server has multiple accounts each will need its own expunge cron job script run as the user that owns the account.

C.2 Synchronizing (or backing up) two IMAP servers

If you have two IMAP server setups and you want to synchronize them you can use the **dsync** command (which belongs to Dovecot) this can either be used to make a complete backup of a IMAP server and it can also be used to synchronize between two IMAP servers.

Example 1. Making a complete backup to a remote server

Make sure that the user exists on the remote server and then make sure that the destination is empty the first time you run the backup command to avoid any conflicts. Assuming the user is “joe” and the remote server is located at IP address `192.168.0.51`.

Login as `root` (on this server, the source) and then give the following command

```
$> dsync -u joe -v backup root@192.168.0.51 -u joe
```

Dsync will now use ssh to login to the remote server and copy all the mails from this server onto the remote server for this user.

Example 2. Synchronizing between two servers

The difference between backup and synchronization is that any changes in the remote server will also be reflected back in the source server. Assuming the same user and server as in the previous example this is now done by as so

```
$> dsync -u joe -v mirror root@192.168.0.51 -u joe
```

Appendix D - Postfix configuration

Below is a complete postfix configuration file. It will setup a mailserver that:

- Assumes the presence of Dovecot for authentication of incoming SMTP connections
- Will relay non-local mail to our ISP's mailserver
- Will replace the local user names with proper mail addresses at our ISP
- Will allow clients to connect using SSL/TLS but will not require it
- Will listen on all interfaces

- Unknown local users will be rejected

Keep in mind that this is a minimal (but working) configuration file and that there are many more parameters that can be adjusted. All parameters that has not been explicitly defined in this file will take on there default values.

D.1 main.cf

```
# Example of a minimal postfix configuration file

# LOCAL PATHNAME INFORMATION
#
# The queue_directory specifies the location of the Postfix queue.
# This is also the root directory of Postfix daemons that run chrooted.
# See the files in examples/chroot-setup for setting up Postfix chroot
# environments on different UNIX systems.
#
queue_directory = /var/spool/postfix

# The command_directory parameter specifies the location of all
# postXXX commands.
#
command_directory = /usr/sbin

# The daemon_directory parameter specifies the location of all Postfix
# daemon programs (i.e. programs listed in the master.cf file). This
# directory must be owned by root.
#
daemon_directory = /usr/lib/postfix

# The data_directory parameter specifies the location of Postfix-writable
# data files (caches, random numbers). This directory must be owned
# by the mail_owner account (see below).
#
data_directory = /var/lib/postfix

# QUEUE AND PROCESS OWNERSHIP
#
# The mail_owner parameter specifies the owner of the Postfix queue
# and of most Postfix daemon processes. Specify the name of a user
# account THAT DOES NOT SHARE ITS USER OR GROUP ID WITH OTHER ACCOUNTS
# AND THAT OWNS NO OTHER FILES OR PROCESSES ON THE SYSTEM. In
# particular, don't specify nobody or daemon. PLEASE USE A DEDICATED
# USER.
#
mail_owner = postfix

# INTERNET HOST AND DOMAIN NAMES
#
# The myhostname parameter specifies the internet hostname of this
# mail system. The default is to use the fully-qualified domain name
# from gethostname(). $myhostname is used as a default value for many
# other configuration parameters.
#
# THE NAME SHOULD BE THE FULLY QUALIFIED NAME OF YOUR SERVER
myhostname = myserevr.mydomain

# The unknown_local_recipient_reject_code specifies the SMTP server
# response code when a recipient domain matches $mydestination or
# ${proxy,inet}_interfaces, while $local_recipient_maps is non-empty
# and the recipient address or address local-part is not found.
#
# The default setting is 550 (reject mail) but it is safer to start
# with 450 (try again later) until you are certain that your
# local_recipient_maps settings are OK.
```

```
#
unknown_local_recipient_reject_code = 550

# TRUST AND RELAY CONTROL

# The mynetworks parameter specifies the list of "trusted" SMTP
# clients that have more privileges than "strangers".
#
# In particular, "trusted" SMTP clients are allowed to relay mail
# through Postfix. See the smtpd_recipient_restrictions parameter
# in postconf(5).
#
mynetworks_style = subnet

# INTERNET OR INTRANET

# The relayhost parameter specifies the default host to send mail to
# when no entry is matched in the optional transport(5) table. When
# no relayhost is given, mail is routed directly to the destination.
#
# In the case of SMTP, specify a domain, host, host:port, [host]:port,
# [address] or [address]:port; the form [host] turns off MX lookups.
#
#
relayhost = [YOUR.ISP.SMTPSERVER]:YOUR.ISP:SMTP.PORT

# ALIAS DATABASE
#
# The alias_maps parameter specifies the list of alias databases used
# by the local delivery agent. The default list is system dependent.
#
# On systems with NIS, the default is to search the local alias
# database, then the NIS alias database. See aliases(5) for syntax
# details.
#
# If you change the alias database, run "postalias /etc/aliases" (or
# wherever your system stores the mail alias file), or simply run
# "newaliases" to build the necessary DBM or DB file.
#
# It will take a minute or so before changes become visible. Use
# "postfix reload" to eliminate the delay.
#
alias_maps = hash:/etc/aliases

# The mailbox_command parameter specifies the optional external
# command to use instead of mailbox delivery. The command is run as
# the recipient with proper HOME, SHELL and LOGNAME environment settings.
# Exception: delivery for root is done as $default_user.
#
mailbox_command = /usr/bin/procmail

# SHOW SOFTWARE VERSION OR NOT
#
# The smtpd_banner parameter specifies the text that follows the 220
# code in the SMTP server's greeting banner. Some people like to see
# the mail version advertised. By default, Postfix shows no version.
#
# You MUST specify $myhostname at the start of the text. That is an
# RFC requirement. Postfix itself does not care.
#
smtpd_banner = $myhostname ESMTP $mail_name
#smtpd_banner = $myhostname ESMTP $mail_name ($mail_version)

# DEBUGGING CONTROL
#
# The debug_peer_level parameter specifies the increment in verbose
# logging level when an SMTP client or server host name or address
# matches a pattern in the debug_peer_list parameter.
```

```

#
debug_peer_level = 2

# The debug_peer_list parameter specifies an optional list of domain
# or network patterns, /file/name patterns or type:name tables. When
# an SMTP client or server host name or address matches a pattern,
# increase the verbose logging level by the amount specified in the
# debug_peer_level parameter.
#
#debug_peer_list = 127.0.0.1
#debug_peer_list = some.domain

# The debugger_command specifies the external command that is executed
# when a Postfix daemon program is run with the -D option.
#
# Use "command .. & sleep 5" so that the debugger can attach before
# the process marches on. If you use an X-based debugger, be sure to
# set up your XAUTHORITY environment variable before starting Postfix.
#
debugger_command =
    PATH=/bin:/usr/bin:/usr/local/bin:/usr/X11R6/bin
    ddd $daemon_directory/$process_name $process_id & sleep 5

# If you can't use X, use this to capture the call stack when a
# daemon crashes. The result is in a file in the configuration
# directory, and is named after the process name and the process ID.
#
# debugger_command =
# PATH=/bin:/usr/bin:/usr/local/bin; export PATH; (echo cont;
# echo where) | gdb $daemon_directory/$process_name $process_id 2>&1
# >$config_directory/$process_name.$process_id.log & sleep 5
#
# Another possibility is to run gdb under a detached screen session.
# To attach to the screen session, su root and run "screen -r
# <id_string>" where <id_string> uniquely matches one of the detached
# sessions (from "screen -list").
#
# debugger_command =
# PATH=/bin:/usr/bin:/sbin:/usr/sbin; export PATH; screen
# -dmS $process_name gdb $daemon_directory/$process_name
# $process_id & sleep 1

# INSTALL-TIME CONFIGURATION INFORMATION
#
# The following parameters are used when installing a new Postfix version.
#
# sendmail_path: The full pathname of the Postfix sendmail command.
# This is the Sendmail-compatible mail posting interface.
#
sendmail_path = /usr/sbin/sendmail

# newaliases_path: The full pathname of the Postfix newaliases command.
# This is the Sendmail-compatible command to build alias databases.
#
newaliases_path = /usr/bin/newaliases

# mailq_path: The full pathname of the Postfix mailq command. This
# is the Sendmail-compatible mail queue listing command.
#
mailq_path = /usr/bin/mailq

# setgid_group: The group for mail submission and queue management
# commands. This must be a group name with a numerical group ID that
# is not shared with other accounts, not even with the Postfix account.
#
setgid_group = maildrop

# html_directory: The location of the Postfix HTML documentation.

```

```

#
html_directory = /usr/share/doc/packages/postfix-doc/html

# manpage_directory: The location of the Postfix on-line manual pages.
#
manpage_directory = /usr/share/man

# readme_directory: The location of the Postfix README files.
readme_directory = /usr/share/doc/packages/postfix-doc/README_FILES

# Notify old style biff Unix clients when new mail arrives default=yes)
biff = no

# The time after which the sender receives the message headers of mail that is still queued.
delay_warning_time = 1h

# Mail aliases
canonical_maps = hash:/etc/postfix/canonical
relocated_maps = hash:/etc/postfix/relocated
sender_canonical_maps = hash:/etc/postfix/sender_canonical
transport_maps = hash:/etc/postfix/transport

# Standard mail directory
mail_spool_directory = /var/mail
message_strip_characters = \0

# No limits on mailboxes of messages
mailbox_size_limit = 0
message_size_limit = 0

# External entropy source for tlsmgr
tls_random_source = dev:/dev/urandom

# SASL configuration for outgoing mail.
# This include telling postfix where to find the login information to the
# remote server being used to relay mails (in sasl_passwd)
smtp_sasl_auth_enable = yes
smtp_sasl_security_options =
smtp_sasl_password_maps = hash:/etc/postfix/sasl_passwd

smtp_use_tls = no
smtp_tls_loglevel = 0
smtp_enforce_tls = no
smtp_tls_CAfile =
smtp_tls_CApolicy =
smtp_tls_cert_file =
smtp_tls_key_file =

# Incoming server settings.
# Among other things this specifies who are allowed to send mails through this
# server and how the authentication of user should be handled.
smtpd_sender_restrictions = hash:/etc/postfix/access
smtpd_recipient_restrictions = permit_mynetworks,permit_sasl_authenticated,reject_unauth_destination

# Allow authentication of incoming connections through SASL
smtpd_sasl_auth_enable = yes

# Uncomment the following line if you expect Outlook express (broken) clients to be able to connect
#broken_sasl_auth_clients = yes

# We use dovecot as SASL provider
smtpd_sasl_type = dovecot
smtpd_sasl_path = private/auth

# Require TLS connections (we will reject non encrypted SASL authentication connections)
smtpd_tls_security_level = may
smtpd_tls_loglevel = 1
smtpd_tls_cert_file = /etc/ssl/certs/mail_public.pem

```



```
smtpd_tls_key_file = /etc/ssl/private/mail_private.pem
smtpd_tls_auth_only = yes

# EOF
```

D.2 master.cf

For a setup that uses port 25 both to received encrypted and non-encrypted communication it is enough to just uncomment the following line in the default master.cf

```
tlsmgr      unix      -        -        n       1000?    1       tlsmgr
```

To enable either the submission port (587) or the (obsoleted) SMTPS port (465) it is necessary to adjust master.cf so that SMTPD listens on that port.

Appendix E - Create SSL/TLS keys and certificates with openssl

In the following we will create our own CA (Certification agency) that will sign all our public keys. This means that to avoid the warnings in browsers about untrusted certificate the CA certificate (our root certificate) should be imported into the browsers. As an example we will show how this is done for the Chrome browser at the end of this appendix.

The main purpose of this section is to show how the bare bones “openssl” command can be used to create certificates rather than relying on the simple but somewhat magical utility scripts we used in the main discussion. We recommend that one you have gotten over the initial chock of dealing with certificates this is actually a simpler method.

As convention we will use the extension *.pem for all keys (since this is the format they keys are in). To distinguish between the public (the certificate) and private key we will add “_public” and “_private” to the main name of the key-pair.

Some documentation uses the suffix “*.crt” and “*.key” for the public and private key in PEM format. However, our opinion is that the suffix should reflect the format of the file (in this case the PEM) format and the use of crt and key as suffixes make things unnecessary complicated. Especially since one might believe that “*.crt” and “*.key” are in themselves are new formats.

At the end of this walk-through we will have to generate the following key-pairs to use in our setup

CA_public.pem, CA_private.pem	This is our own “Root” certificate. This public key is implicitly trusted, i.e. it is trusted because we say so. No higher authority (but us) has sanctioned this key pair.
----------------------------------	---

https_public.pem, https_private.pem	This is the keypair we will use with Apache when https connections are made
--	---

mail_public.pem, mail_private.pem	This is the keypair that will be used for both Dovecot2 as well as postfix to encrypt both IMAP and SMTP protocols into IMAPS and SMTPS
--------------------------------------	---

Optionally we might also have

https_alt_public.pem, https_alt_private.pem	This is a keypair which makes it possible to access the server under several aliases. This if further described in section E.5
--	--

In the process of generating these targets there will also be some intermediate files that strictly speaking is not necessary to keep (but might come in handy later on as we shall see).

The process is now as follows

1. Create the CA key-pair as a root certificate, i.e. it will be “signed” by itself. It is trusted because we say so!
2. Handle the key-pair for https
 - a. Create the key-pair, private and public key
 - b. Prepare the public key to be signed by creating the “Certificate signing request” (CSR) based on the public key
 - c. Sign the public key with our CA private key
3. Handle the key-pair for dovecot and postfix
 - a. Create the key-pair, private and public key
 - b. Prepare the public key to be signed by creating the “Certificate signing request” (CSR) based on the public key
 - c. c. Sign the public key with our CA private key

Please read through the above steps again and make sure you understand the principle. (In practice the creation of the key-pair and the signing request is done in one step – but logically they are separate.)



Caution

There is no shortage of description on the net on how to create so called self-signed certificates. However many of them are dated and based on very old ways of doing things which have just been inherited out of laziness and without proper understanding of the tools involved.

For example, many descriptions do not add the “-nodes” flag as we do in these examples. This will force a password to be created for the private key in certificates and since this is usually not wanted (as described below) one extra step is then required to remove the password. Hence making the description unnecessarily complicated. Another example of obsoleted usage is the option “nsCertType = server” in config files. This option is since long superseded and is not even necessary in our context. Again, adding complexity where none is needed. Another (very popular) complexity adder is to create the private key separately from the certificate request with another obsoleted command; “openssl genrsa”.

Yet another plain wrong way that many HowTo:s show is to create the initial serial number for the certificates in the database (that “openssl ca” keeps of all signed certificates) by simply storing a number one in a file (usually by “echo 01 > serial”). Instead the option “-create_serial” should be given to let openssl initialize a random 128 bit serial number.

Dealing with SSL/TLS certificate introduces many new terms and takes a while getting used to especially since “openssl” is not exactly easy to use. It has evolved (together with the standards) for a long time and old options that are since long superseded remains. However, there is no need to make things more black magic than necessary and the way we show below is what we believe the absolute easiest way to accomplish what we need in the minimum number of steps.

E.1 The use of the term “self-signed certificates”

The term self-signed certificate has two distinct meanings. It could mean that we sign a certificate that we have generated our self with our own CA (root) certificate that we have either created (most likely) or bought (highly unlikely). This is the way we use the term self-signed certificate in this appendix.

It could also mean that we create a CA-type certificate which is (almost) a regular certificate that it not signed by anyone but have a flag inside it that says “trust me, because I say so – I’m a root certificate”. This is really what openssl means by self-signing, the certificate is signed by itself.

Many of the tools out there create this latter type of certificate to avoid having to create a separate CA (root certificate)

E.2 Preparations

When reading the description below you need to watch out for the order of the argument to openssl. Sometimes it doesn't matter but sometimes it does and the error message you get will not really be helpful. For example, calling "openssl req -new -nodes . . ." will work but calling "openssl req -nodes -new . . ." will not work and fail with an error message about unrecognized option "-nodes". While logically this could be explained it's not really helpful if you are not used to the idiosyncrasies of openssl. While we could use the default "/etc/ssl" as our working directory we do not recommend this. Instead we create a pristine directory with our own setup so we won't disturb anything else. Later on it is also easy to create a safety backup of this directory. We will therefore create a new directory which we will use to hold all keys and intermediate files. Since openssl keeps track of all certificates we have signed by means of a simple textual database (and a serial number) we must also setup those initial helper files.

In this directory we will have four additional subdirectories:

- "private", Stores all private keys we create
- "public", Stores all public keys (signed certificates)
- "csr", Stores all certificate signing requests (public keys which have not yet been signed)
- "newcerts", Stores a copy all our signed certificates (the "openssl ca" command will automatically store a copy of all signed certificate in this directory).

Start by creating this directory structure as (we will name our root directory "ca"):

```
$> mkdir ca
$> cd ca
$> mkdir private
$> mkdir public
$> mkdir csr
$> mkdir newcerts
```

Now we create an empty file which later on will hold the the simple text database which "openssl ca" uses to keep track of all certificates that we sign (openssl will complain if this file doesn't exist).

```
$> touch index.txt
```

We shall now create a config file for openssl. One way to do this would be to take the supplied standard openssl config file (in /etc/ssl) and modify a few lines. However, we will create our own smaller version so that we now the exact meaning of every single line we have in the config file. This config file will also have some useful defaults to save us some typing each time we create (or sign) a certificate. We will therefore create our own minimal "our_openssl.cnf" as follows (you should probably name this file after your site and remember to use that name in the commands that follows for clarity).

```
#####
# Simplified openssl config file based on distributed openssl file
# Version 1.0
#####

#####
# CA Definitions - point to section where all default values can be
# found.
[ ca ]
default_ca = CA_default

#####
```

```

# "openssl ca" default parameters
[ CA_default ]

dir = .
new_certs_dir = $dir/newcerts
database = $dir/index.txt
certificate = $dir/public/ca_public.pem      # Our CA (root) certificate (public key)
private_key = $dir/private/ca_private.pem    # The private key for our CA key-pair
serial = $dir/serial

# In our case we will not really be using Certificate Revocation Lists
# but we include the setup for future use
crl_dir = $dir/crl
crl_number = $dir/crlnumber
crl = $crl_dir/crl.pem

default_days = 3650      # We sign certificates that are valid for ~10 years
default_md = sha1        # The sha1 encryption method is recommended over the older md5
name_opt = ca_default
cert_opt = ca_default
preserve = no
policy = policy_match

x509_extensions = usr_cert
copy_extensions = copy   # Honor the certificate

#####
# The default policy for the CA when signing requests We only request
# that certificate is from the same country
[ policy_match ]
countryName             = match
stateOrProvinceName     = supplied
organizationName        = supplied
organizationalUnitName  = supplied
commonName              = supplied
emailAddress            = supplied

#####
#Default values for use with "openssl req" command
[ req ]
default_bits = 2048
default_md = sha1
string_mask = utf8only
distinguished_name = req_distinguished_name
req_extensions = v3_req      # Extensions to add when creating certificate requests
x509_extensions = v3_ca      # Extensions to add to self-signed certificates

#####
# Prompts and default values for the fields in the DN
[ req_distinguished_name ]
0.organizationName = Organization Name
0.organizationName_default = Aditus

organizationalUnitName = Unit or department in organization
organizationalUnitName_default = IT-Adm

emailAddress = Email Address
emailAddress_max = 64
emailAddress_default = admin@aditus.nu

localityName = City or district
localityName_default = Sundbyberg

stateOrProvinceName = State or Province Name
stateOrProvinceName_default = Sundbyberg

countryName = Country Code (2 letter code)
countryName_min = 2

```

```

countryName_max = 2
countryName_default = SE

commonName = Common Name (hostname, IP, or your name)
commonName_max = 64

#####
# Extension values used when signing certificates as a CA
[ usr_cert ]
basicConstraints=CA:false
subjectKeyIdentifier=hash
authorityKeyIdentifier=keyid,issuer

#####
# For the normal case where we do not create CA certificate but only
# ordinary client certificates
[ v3_req ]
basicConstraints=CA:false
subjectKeyIdentifier=hash

# Uncomment the following if you want the certificate request generated on this server
# to have several aliases for the common name. Unfortunately this parameter can not be
# given on the command line.
# (Alternatively this could be specified as "subjectAltName = DNS:example.com,
# DNS:www.example.com" instead as a list as we have done it now.
# subjectAltName = @alt_names

# Alias name for this server when creating signing requests. These are just examples!!
# Uncomment the following section and set proper names
#[ alt_names ]
#DNS.1 = sigma1
#DNS.2 = sigma1.nil

#####
# Section to be added when we give the command line option -extensions v3_ca
# together with the x509 option meaning that this should be a self-signed
# certificate, i.e CA=TRUE
[ v3_ca ]
basicConstraints = CA:TRUE
subjectKeyIdentifier = hash
authorityKeyIdentifier = keyid:always,issuer:always

#EOF

```

E.3 Generating the CA key-pair

We are now in position to generate our root (CA) certificate. The first step is to create the key-pair and the signing request

```
$> openssl req -new -keyout private/ca_private.pem -out csr/ca_public_req.pem \
-config ./our_openssl.cnf
```

The next step is for us to sign it our self

```
$> openssl ca -create_serial -out public/ca_public.pem -selfsign -extensions v3_ca \
-config our_openssl.cnf -days 7000 -infile csr/ca_public_req.pem
```

This will prompt you for a number of things most of them have default values as specified in the config file. Do not forget the password you gave here for the CA private key.

The special options used for the CA signing as compared to all the signings we will do from here on are the two options

- `-selfsign`, Which tells openssl to use the `ca_public_req.pem` when signing instead of expecting a CA certificate (which we haven't created yet)
- `-extension v3_ca`, needed to read the `v3_ca` section in the config file.

Our CA certificate (root certificate) will be valid for roughly 20 years (~7000 days). We have now produced the following files:

- `private/ca_private.pem`, The password protected private key part of our CA-key pair. This file should be kept secret at all times!
- `public/ca_public.pem`, The public key that should be imported into browsers that we want to accept any other public keys (certificates) that we will sign in the future.
- `csr/ca_public_req.pem`, The signing request for our CA
- `serial`, This is an administrative file used by “`openssl ca`” to give a unique serial number for each of the certificate that we sign.
- `newcerts/<random_serial_number>.pem`, A copy of our signed certificate. The exact name is random since it starts with the serial number that was randomly generated. From this moment on each serial number will be incremented by one for each new certificate that we sign.

Many other tutorials show what seems to be a simpler way to create root certificate in only one step using the `-x509` option with openssl. As in

```
$> openssl req -new -x509 -extensions v3_ca -keyout private/ca_private.pem \
-out public/ca_public.pem -days 7000 -config ./our_openssl.cnf
```

However, we do not recommend this way since it has a number of drawbacks:

- You cannot use `-create_serial` with the “`req`” command
- The signing request is not kept
- There is no copy for our signed root certificate in `newcerts` as it is with all other cert we created

This could however be used in another way. If you need to quickly create a “test-certificate” (which isn't signed by any trusted authority or yourself) you could create such a certificate in the same way (no need to include the `-extensions v3_ca` in this case though) such as

```
$> openssl req -new -nodes -x509 -keyout private/tstcert_private.pem \
-out public/tstcert_public.pem -days 7000 -config ./our_openssl.cnf
```

You can now use the resulting certificate if you need to run some quick test with some software that requires a SSL/TLS certificate (key-pair).



Note

Note on x509v3 compliancy. The CA certificate we have created here is, while perfectly working, not formally fully compliant with the very latest version of the PKIX and X509v3 standards since that requires the email address to be specified in a slightly different place. This is however of no practical concern for us (unless you plan on starting your own worldwide certification agency and charge money for signing certificates)

E.4 Generating the Apache HTTPS key-pair

We will now generate the key-pair that will allow HTTPS traffic. We will show two ways of doing this. The first (and simplest) is to just generate a key-pair which is valid for exactly one common name, i.e. the name you access the browser with. On a home network where you probably don't have a DNS server you will most likely access the server via an IP address directly. The problem with only having one name in the certificate is that if you access the browser through some other methods (perhaps from outside your router with a completely different IP address) you might again get a warning about untrusted site even if you have imported browser public key.

The following command will generate a key-pair. As you can see we have added the suffix “_req” to the public key. That is to remind ourselves that this will just be used as a certificate signing request, i.e. this public key has not yet been signed by our CA key-pair created in the previous step. After we have signed the key it will be named “https_public.pem”

```
$> openssl req -new -nodes -keyout private/https_private.pem \
-out csr/https_public_req.pem -config ./our_openssl.cnf
```

Answer all the questions that you are now presented with. Keep in mind that every certificate must have a unique distinguished name that is made up of the fields like company name, city, department/unit etc. A recommendation is therefore to use “Web” (or Apache or similar) as the unit name to differentiate this certificate from the other mail certificate we need to create as a next step.

For future reference you should save all requests and keys. The request can be re-used if the certificate has expired. So far we produced the following new files:

- `csr/https_public_req.pem`, This is the public key that we will sign with our CA keys
- `private/https_private.pem`, This is our private key that will be loaded by Apache

It is now time to sign the public key with our CA key. This is done with the command

```
$> openssl ca -out public/https_public.pem -config ./our_openssl.cnf \
-infiles csr/https_public_req.pem
```

Note: The config file has to be given on the command line before the “-infiles” command!

Answer the questions and use the password for the private CA key we created in section E.3. We have now generated the files

- `public/https_public.pem`, This is the signed public certificate for Apache
- `newcerts/<some_large_serial_number>.pem`, A copy for our own record of the just created certificate. The naming is the serial number of the certificate.

This newly signed certificate is also recorded in our database file “index”

E.5 Generating the Dovecot/postfix key-pair

This is done exactly analogue (but with the obvious change of filenames) as when we created the HTTPS key-pair. The commands to create the key-pair are the following.

First create the signing request

```
$> openssl req -new -nodes -keyout private/mail_private.pem \
-out csr/mail_public.pem -config ./out_openssl.cnf
```

And then signing this request with our own root certificate

```
$> openssl ca -out public/mail_public.pem -config ./our_openssl.cnf \
-infiles csr/mail_public_req.pem
```

These lines have now created the following files

- `private/mail_private.pem`, The private key for our mail setup
- `public/mail_public.pem`, The public key for our mail setup
- `csr/mail_public_req.pem`, The signing request. This is not needed any more but could be kept to re-generate the certificate when it expires in 10 years.
- `newcerts/02.pem`, A copy of the signed certificate for our records as a certification agency

E.6 Generating a certificate with multiple common names

As mentioned above most people create certificates as we did above. However, for the case the same server is accessed through many names (for example “www.myserver.com” or “myserver.com” or “199.243.243.12” etc) this certificate will still generate a warning if we access the server with any other name besides what we used when we created the certificate signing request. To create a certificate with multiple alternative names requires some additional setup. In the preparation section our “openssl.cnf” contains all the necessary parameters but we need to uncomment them. First uncomment the line

```
copy_extensions = copy
```

Then uncomment and setup the lines at the end of the file

```
subjectAltName = @alt_names

# Alias name for this server when creating signing requests.
# These are just examples!!
# Uncomment the following section and set proper names
[alt_names]
DNS.1 = example.com
DNS.2 = www.example.com
DNS.3 = 199.199.199.199
```

To match the values you want to have. The you can just generate the certificate as in the same way we did before (we just use a different name for the certificate to keep them apart)

```
$> openssl req -new -nodes -keyout private/https_alt_private.pem \
-out csr/https_alt_public_req.pem -config ./openssl.cnf
```

Note: You must change some entry in the DN fields compared to the previously created HTTPS certificate since our Database for certificates we have signed must all have unique DN. You could for example change your CN (Common Name).

We can view the request to make sure that we have the alternative names included with the command.

```
$> openssl req -in csr/https_alt_public_req.pem -noout -text
```

and in the long output the following lines can be seen

```
Requested Extensions:
```



```
X509v3 Basic Constraints:  
    CA:FALSE  
X509v3 Subject Key Identifier:  
    7A:2E:34:23:E5:EB:D3:15:56:9F:8E:D9:07:13:6C:7D:14:40:E1:B2  
X509v3 Subject Alternative Name:  
    DNS:199.199.199, DNS:example.com, DNS:www.example.com  
    . . .
```

The ones marked with red states the alternative names that can be used with this certificate. It is now time to sign it the certificate

```
$> openssl ca -out public/https_alt_public.pem -config ./openssl.cnf \  
-infiles csr/https_public_req.pem
```

And after this command we have the following new files

- public/https_alt_public.pem
- private/https_alt_private.pem
- csr/https_public_req.pem
- newcerts/03.pem

E.7 Keeping both the certificate and key in the same file

It is useful to know that if some servers only give you the option to specify one SSL/TLS file you must combine both the public (certificate) and the private key in one file. You can easily do this as

```
$> cat private/mail_private.pem public/mail_public.pem > mail_combined.pem
```

You could also do this for Dovecot and postfix since they will read the correct key based on the headers in the file. Some might think that this is better way of keeping out key-pair since we can never lose one key in the key-pair if we keep them combined.

E.8 Converting to other certificate formats

As mentioned previously there are other variant formats besides PEM used to store certificates and keys. More specifically there are two other formats commonly used as described below.

E.8.1 DER format

File suffix: *.der

DER stands for Distinguished Encoding Rules. This format is also quite often used on MS Windows system as well as with Java. This is a binary format so it cannot directly be viewed as a text file (in opposite from the pem format).

Convert an existing certificate in PEM format to the equivalent DER format is straight forward using openssl

```
$> openssl x509 -inform pem -in certificate.pem -outform der \
-out certificate.der
```

E.8.2 PKCS #12 format

File suffix: *.p12

PKCS stands for [Definition: Public-Key Cryptography Standards] and #12 is a format in which both a private and a public key is stored in the same file for the purpose of authentication.

The PKCS format is part of a larger set of standards defined by the RSA Laboratories [<http://www.rsa.com/rsalabs/node.asp?id=2124>]. Some of these standard are also defined as RFCs. This specific standard (#12) is a format used to store client certificates. Client certificates are used in analogy to server certificates but to identify the client and not a server. A server can for example be setup in such a way that a client must have a trusted certificate in order to connect to the server. The server trusts a specific certificate if the certificate have been signed by an authority accepted by the server.

It would bring to far to go through all the details of this format here. However this type of format is handled by the pkcs12 subcommand to **openssl** command.

To create a client certificate to be used for authentication based on a existing key-pair use the command

```
$> openssl pkcs12 -export -clcerts -in mypublickey.pem \
-inkey myprivatekey.pem -out myclcert.p12
```

One difference here is that the client certificate will be protected by a password so this will have to be given when the certificate is created (openssl will ask for it).

In order to view some details an existing client certificate use the following command

```
$> openssl pkcs12 -noout -info -in myclcert.p12
Enter Import Password: <SECRET_PASSWORD>
MAC Iteration 2048
MAC verified OK
PKCS7 Encrypted data: pbewithSHA1And40BitRC2-CBC, Iteration 2048
Certificate bag
PKCS7 Data
Shrouded Keybag: pbewithSHA1And3-KeyTripleDES-CBC, Iteration 2048
```

In the output there is a statements about "MAC verified OK". In this context MAC stands for [Definition: Message Authentication Code] and is a way to guarantee that the certificate is valid and has not been tampered with. The reference to PKCS #7 refers to specific message syntax which is detailed in RFC2315 [<http://tools.ietf.org/html/rfc2315>]

E.9 Certification Revocation List (CRL) and OCSP

One thing that we haven't yet touched upon is the directory `crl/` under the apache configuration directory. This directory is used to store a Certificate Revocation List. Basically a list of certificates that the issuer might revoked and should no longer be trusted. From the server perspective this is used to verify that any certificate a client might present is still valid.

The problem with such a revocation list is that it must be constantly renewed to be of any value. In addition both the client and the server needs to have access to an updated revocation lists since neither part can really trust the other part.

For the reason with the need of constant updates of the revocation lists this has too large extent been replaced by online direct checking using what is known as OCSP (Online Certificate Status Protocol). Basically the client will verify any certificate given by the server by going online and check a central revocation lists on a dedicated certificate server.

Ideally all official root certificates (CA certificates) will have ps59D7005DKsuch a server. So for example if a server claims to have a certificate verified by, say, Verisign the client will go online to the Verisign OCSP server and double check that the certificate is really still valid.

We will not discuss this any further in this tutorial but should the need to setup a certificate server arise the open source server "OpenCA" (<http://www.openca.org/>) server might be good candidate to examine since it also includes a OCSPD server daemon.

E.10 Summary

With the above steps we have created the key-pairs we need for both the Web-server as well as the mail servers (Dovecot2 and postfix). You should now take a backup of this directory and keep it in a safe place. When you later on upgrade your servers you should restore these certificates to avoid client warnings.

Since there is quite a lot of text in the previous section we just repeat the actual commands below that we used to create the key-pairs to make it clearer that it is not that difficult. Each command set first creates the key-pair and then signs it.



Caution

Remember to uncomment and set the "subjectAltName" in the configuration file if you want your certificates to be valid for multi-homed servers, i.e. servers that are accessed with several different DNS names (but only one IP-address).

E.10.1 Creating the CA (root certificate)

```
$> openssl req -new -keyout private/ca_private.pem \
-out csr/ca_public_req.pem -config ./our_openssl.cnf
$> openssl ca -create_serial -out public/ca_public.pem \
-selfsign -extensions v3_ca -config our_openssl.cnf \
-days 7000 -infiles csr/ca_public_req.pem
```

E.10.2 Creating the Apache HTTPS certificate

```
$> openssl req -new -nodes -keyout private/https_private.pem \
-out csr/https_public_req.pem -config ./our_openssl.cnf
$> openssl ca -out public/https_public.pem -config ./our_openssl.cnf \
-infiles csr/https_public_req.pem
```

E.10.3 Creating the mail certificate

```
$> openssl req -new -nodes -keyout private/mail_private.pem \
-out csr/mail_public_req.pem -config ./our_openssl.cnf
$> openssl ca -out public/mail_public.pem -config ./our_openssl.cnf \
-infiles csr/mail_public_req.pem
```

Appendix F - Apache virtual host example configuration

The following example configuration shows a complete virtual host configuration for a HTTPS server. The configuration assumes that the host is at IP address 192.168.0.50. You should also notice that this setup relaxes the overly restrictive default settings to allow symbolic links and CGI programs to be executed.

Remember also that in addition to the virtual host configuration file the `/etc/sysconfig/apache2` configuration also needs to be updated as described in a previous chapter to allow HTTPS traffic.

F.1 ssl-vhost.conf

The following shows an example of a complete virtual host configuration



Note

The initial `<VirtualHost _default_:443>` will make these settings apply on *all IP addresses that no other virtual host is bound to*. Please observe that this is quite different from `<VirtualHost *:443>` which will match *all IP addresses regardless if they are used by other virtual host or not*.

The matching is valid only for numeric IP add not DNS names.

```
<VirtualHost _default_:443>

# General setup for the virtual host
DocumentRoot "/srv/www/vhosts/ssl-htdocs"
ServerName 192.168.0.50:443
ServerAdmin root@localhost
ErrorLog /var/log/apache2/ssl-htdocs.err
TransferLog /var/log/apache2/ssl-htdocs.access
CustomLog /var/log/apache2/ssl-htdocs.info ssl_combined

DirectoryIndex index.html

RewriteEngine On
RewriteOptions Inherit

HostnameLookups Off
UseCanonicalName Off

ServerSignature On

Include /etc/apache2/conf.d/*.conf

<Directory /srv/www/vhosts/ssl-htdocs>
    AllowOverride All
    Options Indexes FollowSymLinks ExecCGI
    Order allow,deny
    Allow from all
</Directory>

ScriptAlias /cgi-bin/ /srv/www/vhosts/ssl-cgi-bin
<Directory /srv/www/vhosts/ssl-cgi-bin>
    AllowOverride All
    Options +ExecCGI -Includes
    Order allow,deny
```

```
    Allow from all
</Directory>

SSLCertificateFile /etc/apache2/ssl.crt/server.crt
SSLCertificateKeyFile /etc/apache2/ssl.key/server.key
SSLEngine on
SSLProtocol all -SSLv2 -SSLv3
SSLCipherSuite ALL:!aNULL:!eNULL:!SSLv2:!LOW:!EXP:!MD5:@STRENGTH

<Files ~ "\.(cgi|shtml|phtml|php3?)$" >
    SSLOptions +StdEnvVars
</Files>
<Directory "/srv/www/cgi-bin">
    SSLOptions +StdEnvVars
</Directory>

SetEnvIf User-Agent ".*MSIE [1-5].*" \
    nokeepalive ssl-unclean-shutdown \
    downgrade-1.0 force-response-1.0

</VirtualHost>
```

